Steering what a network learns

Inductive bias: building a task's invariances into the data or into the architecture

01

Learning isn't enough: which solution?

02

Building shift into the weights: convolution

03

Does the bias pay? The shift test and data efficiency

04

The price of an assumption

05

Choosing the bias your data has

© 2026 Dr. Jesse Roberts

Quiz next Thursday

A heads-up before we start

THIS THURSDAY · OCT 29

Details released

What the quiz covers and how it will run.

NEXT THURSDAY · NOV 5

Quiz

Full details come out this Thursday.

Details **this Thursday**; quiz **next Thursday**.

© 2026 Dr. Jesse Roberts

Learning isn't enough

Three questions about deep networks, and the one we start on today

1 **Can a network represent it?**

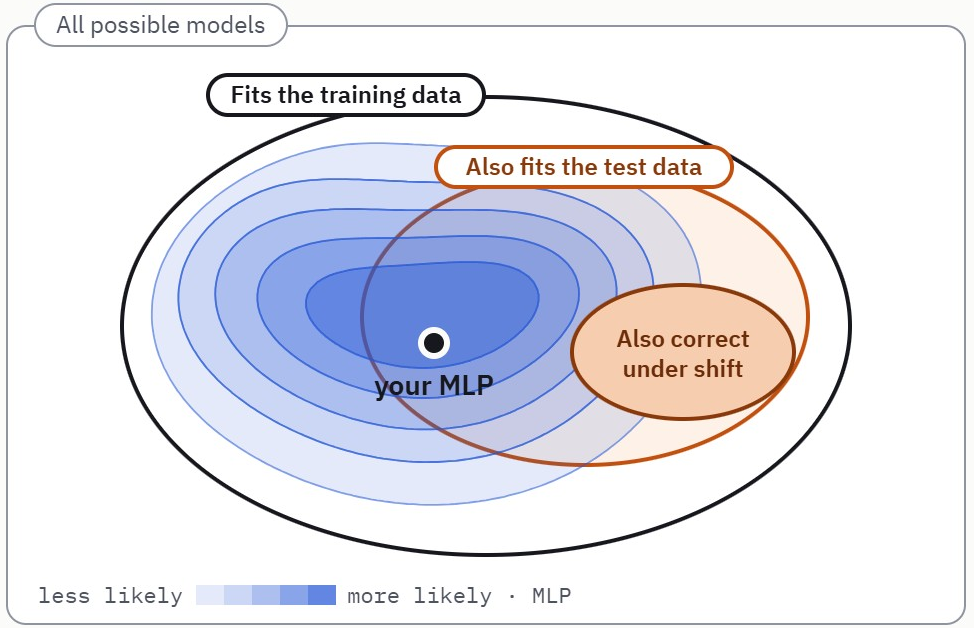
Almost anything. Your MLP even memorized 8,000 random labels.

2 **Can we make it learn?**

Activations that pass gradients, softmax and cross-entropy, backpropagation, optimizers.

3 **Which solution does it learn?**

Many fits exist. We want one that respects the task, and we want it **from as little data as possible**.



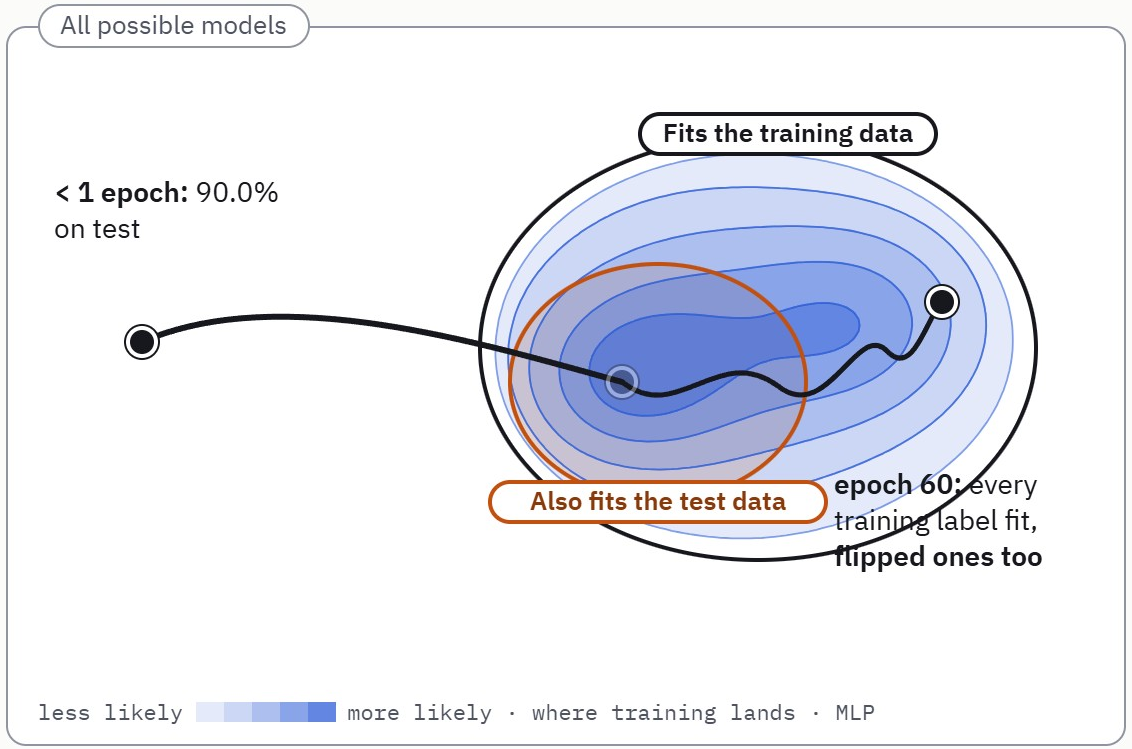
Thursday's map, shaded by where an MLP is likely to land. It landed on a fit that passes the test set but reads a 7 shifted 3 px as a 3.

Today's job is **adjusting which fit** the network lands on.

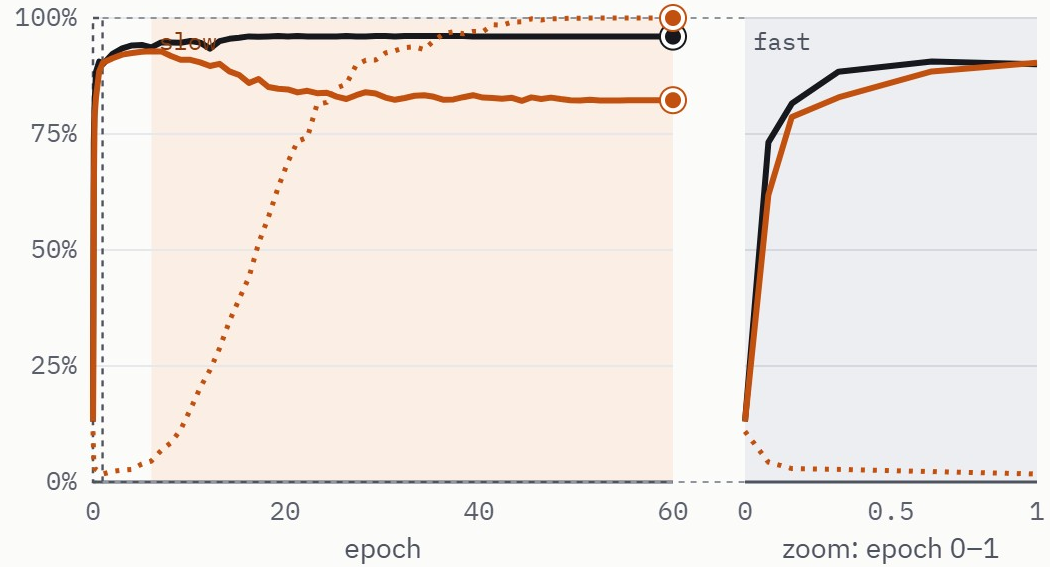
© 2026 Dr. Jesse Roberts

Fast to generalize, slow to memorize

Where SGD takes the MLP through model space, epoch by epoch, on 8,000 training digits



What we measured



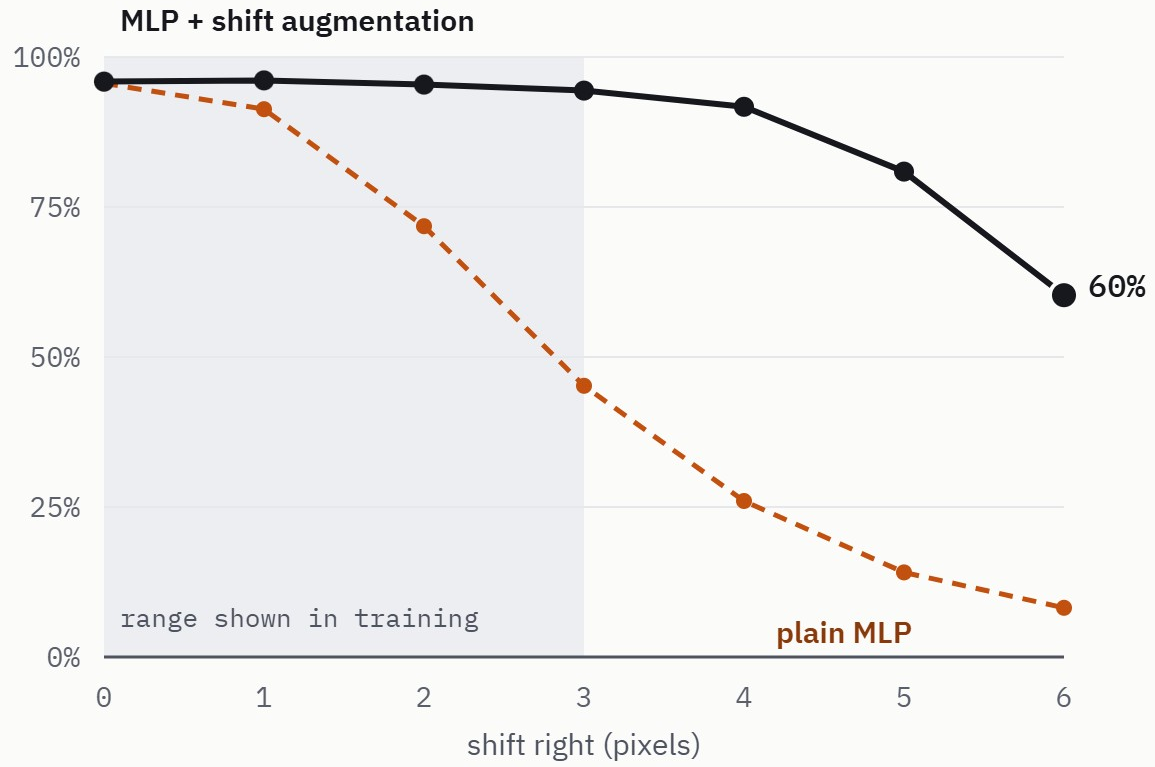
Not the classical picture: with true labels the MLP fits every training digit and test accuracy **still rises** (96.0%). With flipped labels it peaks at 92.8% (epoch 6), then falls to 82.3% as it memorizes them.

SGD finds what generalizes **first**; memorizing what doesn't transfer takes **many more epochs**.

© 2026 Dr. Jesse Roberts

Data augmentation is expensive

Same MLP, same 8,000 digits, retrained with random shifts of up to ±3 px



The bill

Every case must be shown. Shift × scale × rotation is 1,715 versions of every digit: 1.0×10^8 images for MNIST.

Capacity spent on copies. The MLP still stores each stroke separately at every position it was shown.

Only the range shown. 94% at 3 px, but 60% at 6 px, a shift it never saw.

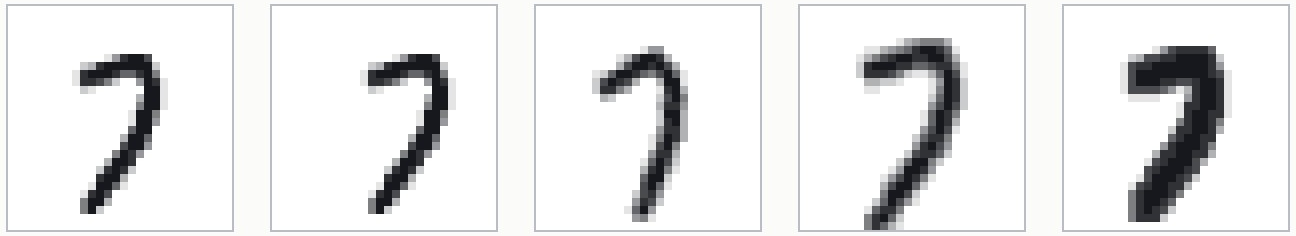
Augmentation teaches an invariance **one example at a time**.

© 2026 Dr. Jesse Roberts

Invariances: changes that keep the label

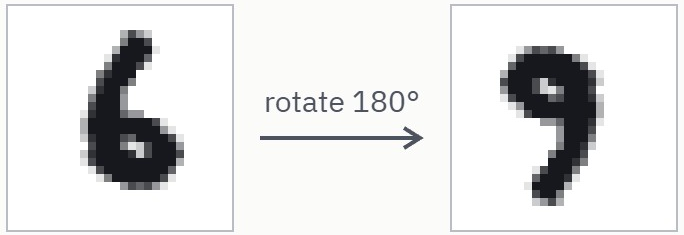
What the network should learn to ignore, written as transformations of the input

Keep the label: still a 7



original | shift 3 px | rotate 15° | scale 1.2× | thicker strokes

Change the label: not an invariance



a 6 → now a 9

Which changes are invariances depends on the **task**: turning a photo of a cat upside down leaves a cat; turning a 6 leaves a 9.

INVARIANCE OF THE TASK

A transformation T is an invariance if it never changes the true label:

$$y(T(x)) = y(x) \text{ for every } x$$

We want the network to share it:

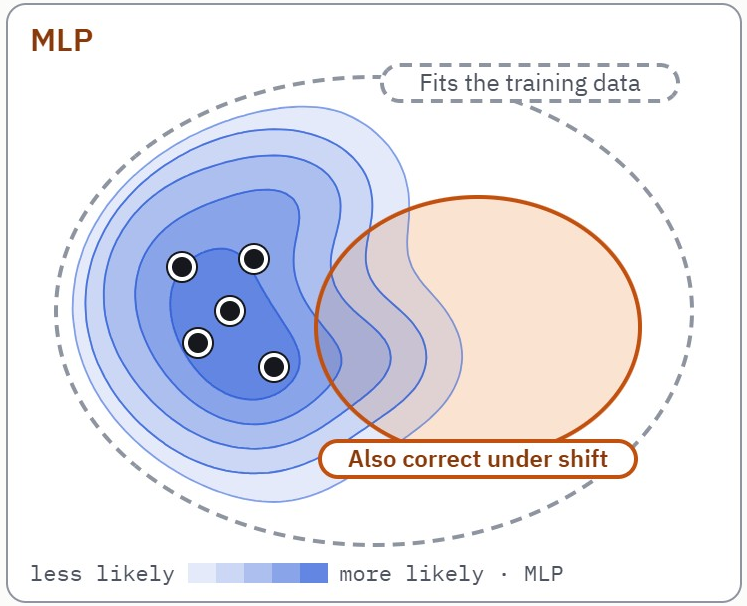
$$\hat{y}(T(x)) = \hat{y}(x)$$

An invariance is knowledge about the task: **what can change without changing the answer**.

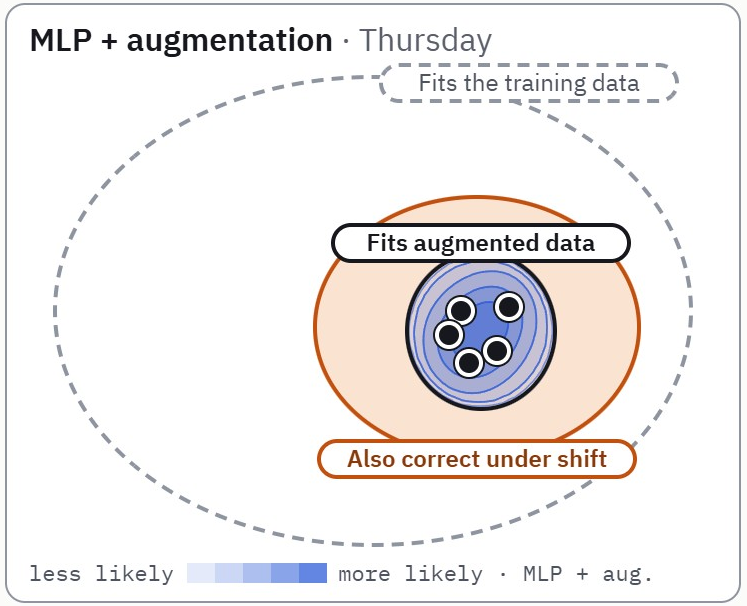
© 2026 Dr. Jesse Roberts

Two paths to learning invariances

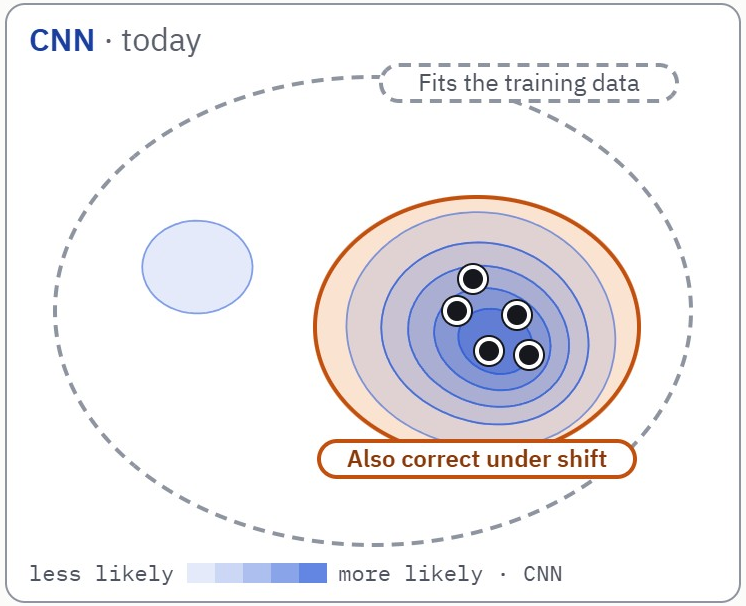
Same space of models, same digits: where does training land?



No help: training lands anywhere that fits, **mostly where shifted digits fail**.



Lever 1, the data. Shifted copies shrink what fits, so training can only land on models that handle them.



Lever 2, the architecture. Same data as the MLP; tied local weights make the shift-correct fits **far likelier**.

Invariance · `of the task`
A change that leaves the label alone: a shifted 7 is still a 7.

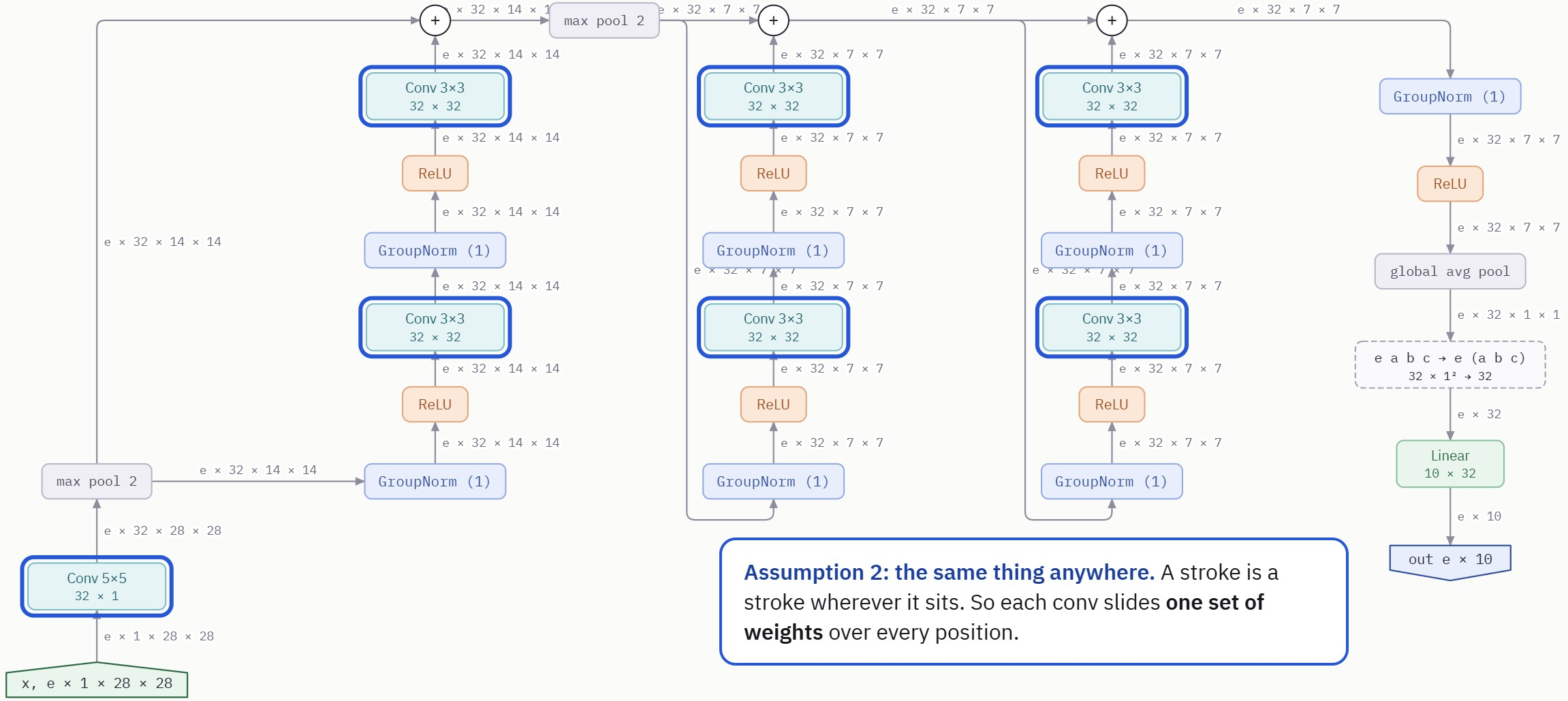
Inductive bias · `of the learner`
Which models training is likely to land on, before any data.

Both levers change **where training is likely to land**, not what a network could represent.

© 2026 Dr. Jesse Roberts

A CNN makes two assumptions

The network we will build today. What makes it a CNN is two bets about images.



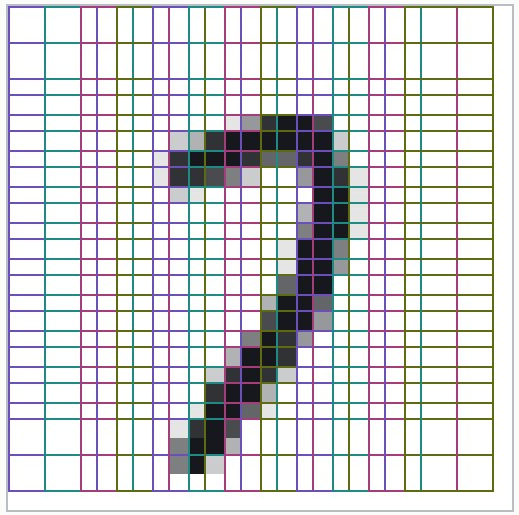
Assumption 2: the same thing anywhere. A stroke is a stroke wherever it sits. So each conv slides **one set of weights** over every position.

© 2026 Dr. Jesse Roberts

Assumption 1: locality

Smallest: 5 × 5 windows, stride 2, one neuron per window

input X (28 × 28)



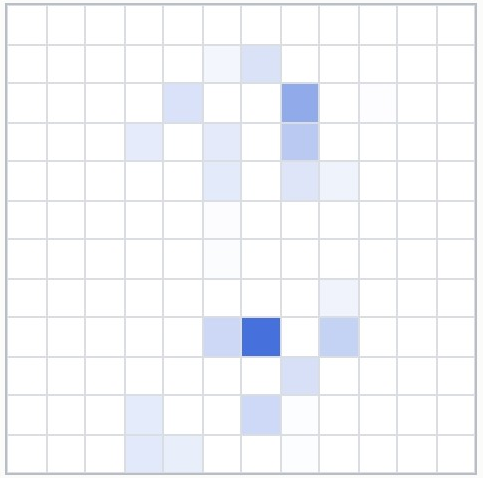
STRONGEST: NEURON 103 OF 144 · WINDOW ROW 8, COLUMN 6

its own 25 weights						pixels under it				
0	+0.6	+0.5	+0.8	+0.5		0.0	0.0	0.3	1.0	1.0
+0.2	+0.3	+0.3	−0.4	+0.7		0.0	0.0	0.8	1.0	1.0
−0.9	−0.1	+0.6	+0.1	+0.8	×	0.0	0.6	1.0	0.9	0.4
+0.9	+0.9	+0.7	+0.4	+0.7		0.3	1.0	1.0	0.9	0.0
+0.4	−0.8	0	+0.7	+0.5		0.9	1.0	0.9	0.2	0.0

Σ → ReLU → **5.0**

Each window is **its own neuron with its own 25 weights**. Stride 2: the next neuron's window starts 2 pixels over.

output (12 × 12): one value per neuron



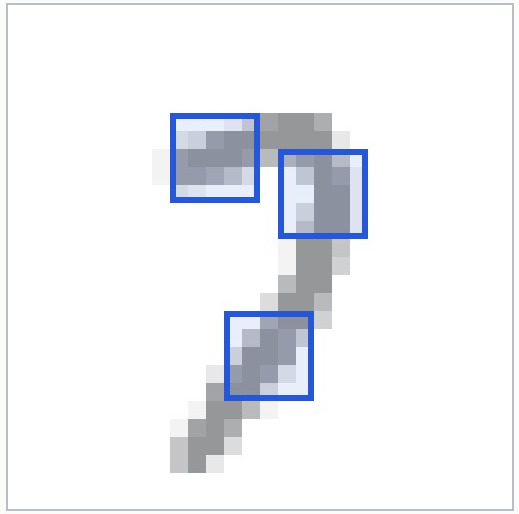
Still has addresses. 144 neurons, 144 separate sets of 25 weights. A stroke one neuron learns is **no help** to its neighbour.

Locality: **some input to some neurons**. Fewer weights each, but every neuron still learns on its own.

© 2026 Dr. Jesse Roberts

Assumption 2: Features are spatially invariant

A stroke anywhere is the same stroke, so every unit can use the same 25 weights



−1	−1	−1	−1	−1
−1	−1	−1	−1	−1
0	0	0	0	0
+1	+1	+1	+1	+1
+1	+1	+1	+1	+1

unit (6, 9)

−1	−1	−1	−1	−1
−1	−1	−1	−1	−1
0	0	0	0	0
+1	+1	+1	+1	+1
+1	+1	+1	+1	+1

unit (8, 15)

−1	−1	−1	−1	−1
−1	−1	−1	−1	−1
0	0	0	0	0
+1	+1	+1	+1	+1
+1	+1	+1	+1	+1

unit (17, 12)

One set of 25 weights, W, used by every unit. A stroke detector learned anywhere works everywhere.

WEIGHTS FOR ONE 24 × 24 MAP OF UNITS

Fully connected	4.5×10^5
Local 5 × 5 patches	1.4×10^4
Local and shared	**25**

THIS IS A CONVOLUTION

$$\hat{y}_{ij} = \Sigma_a \Sigma_b W_{ab} X_{i+a,\, j+b}$$

a, b = 0…4: one filter W, slid to every position (i, j). Deep learning calls it convolution; strictly it is cross-correlation, since W is not flipped.

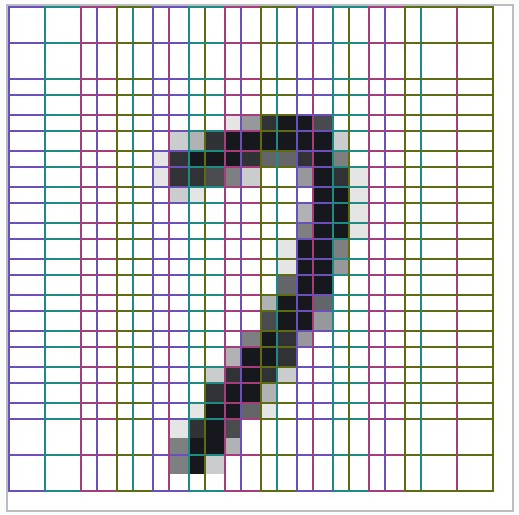
Sharing isn't a trick to save weights. It is the assumption that **a stroke means the same thing everywhere**.

© 2026 Dr. Jesse Roberts

Shared weights for every neuron

The same windows as before, but now every neuron shares one set of 25 weights

input X (28 × 28)



STRONGEST: NEURON 31 OF 144 · WINDOW ROW 2, COLUMN 6

−1	−1	−1	−1	−1
−1	−1	−1	−1	−1
0	0	0	0	0
+1	+1	+1	+1	+1
+1	+1	+1	+1	+1

the same 25 weights W

×

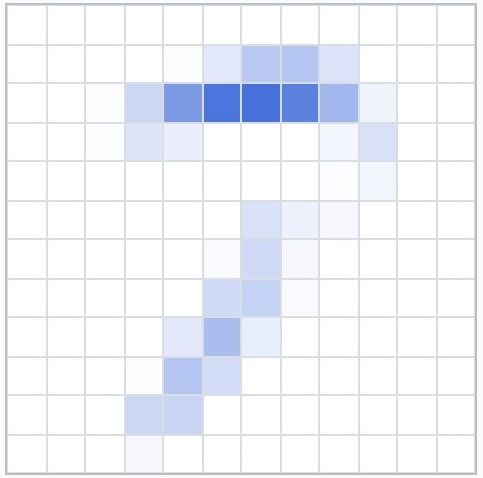
0.0	0.0	0.0	0.0	0.0
0.0	0.0	0.0	0.0	0.0
0.1	0.4	0.9	1.0	1.0
1.0	1.0	1.0	1.0	1.0
1.0	0.9	0.7	0.7	0.9

pixels under it

Σ → ReLU → **9.1**

Every window is a neuron, and **every neuron uses the same W**: −1 above, +1 below, a top-edge detector. Same windows and stride as the last slide.

feature map (12 × 12): one value per neuron



It lights up along the top edges of the 7, **wherever they are**. Nothing in W says where to look.

Tied weights. 144 neurons, **one** set of 25 weights. A stroke detector learned anywhere works everywhere, so the output is a map of where the top edges are. (With stride 1 the map is 24 × 24, as on the next slides.)

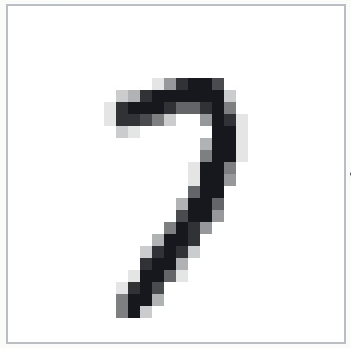
One filter slid everywhere gives a **map of where its stroke appears**.

© 2026 Dr. Jesse Roberts

Shift in, shift out

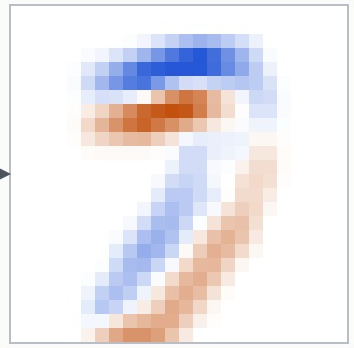
Filter W on Thursday's 7, and on the same 7 shifted 3 px right (the one the MLP read as a 3)

original 7

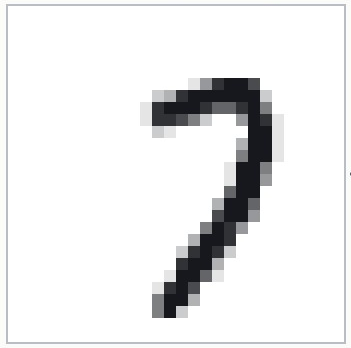


* W

its feature map

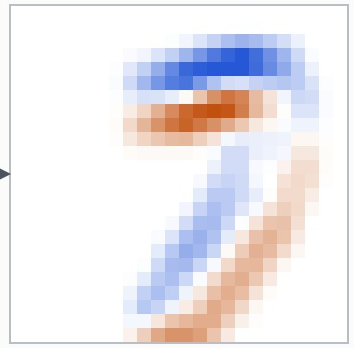


shifted 3 px right

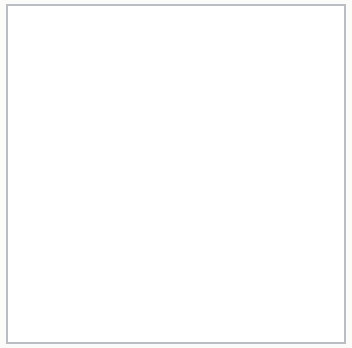


* W

its feature map



bottom map − top map
moved 3 px right



max |difference| = 0.0

EQUIVARIANCE

$W * \text{shift}(X) = \text{shift}(W * X)$

Shift the input and the map shifts by the same amount. **Every value is unchanged**, just moved.

No independent border weights sit untrained. All weights are shared.

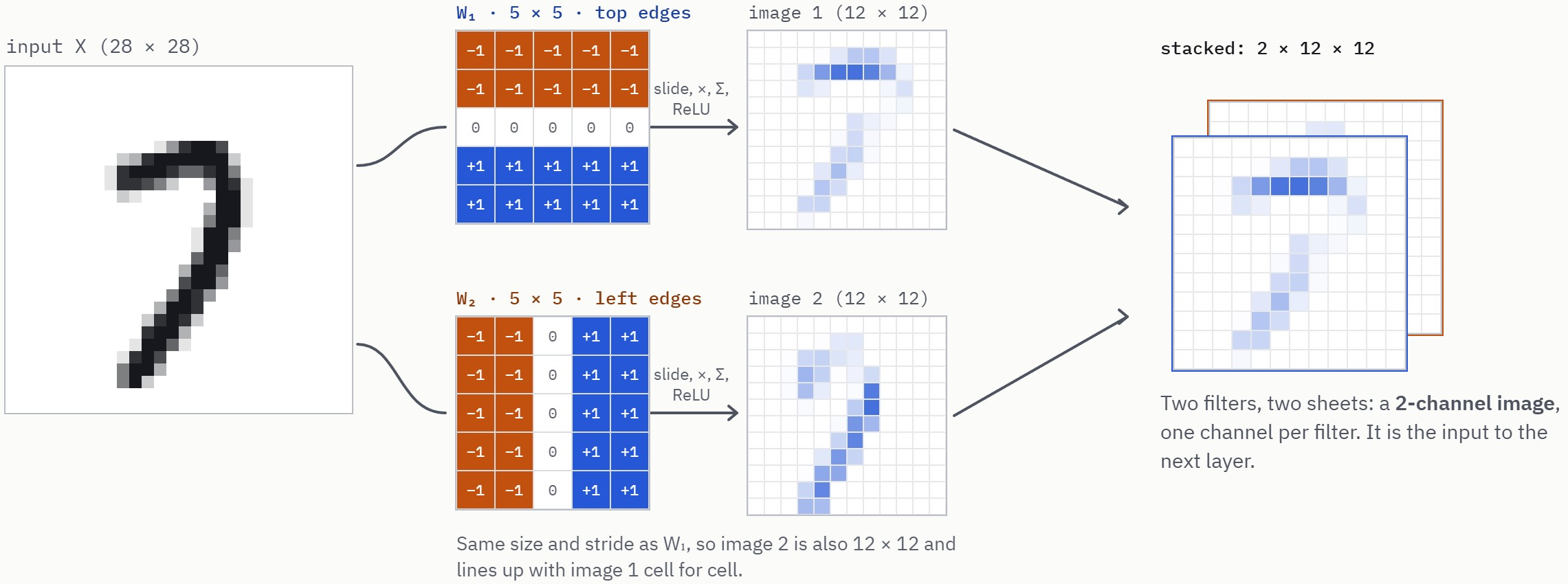
The 25 weights that found the bar at column 12 find it at column 15.

Convolution doesn't ignore position: it **moves the answer with the input**.

© 2026 Dr. Jesse Roberts

Additional filters add additional channels

One filter finds one kind of stroke. Add a second filter and you get a second image of the same 7.



Same size and stride as W_1, so image 2 is also 12 × 12 and lines up with image 1 cell for cell.

Two filters, two sheets: a **2-channel image**, one channel per filter. It is the input to the next layer.

Many filters per layer: each finds its own feature, and **their images stack into channels**.

© 2026 Dr. Jesse Roberts

One Conv2d, 32 filters, 32 channels

The first layer of our network, as set in torch-canvas and as written in PyTorch

Convolution (2-D)

$$y[o, i, j] = b[o] + \Sigma_{c,u,v} W[o, c, u, v] \cdot x[c, s{\cdot}i +$$

A Linear on each 5 × 5 window of all 1 input channels, slid over the image with stride s = 1 and zero padding p = 2; the same weights at every position. W is 32 × 1 × 5 × 5. Output size ⌊(H + 2p − k) / s⌋ + 1.

nn.Conv2d(1, 32, 5), weight 32 × 1 × 5 × 5 = 800

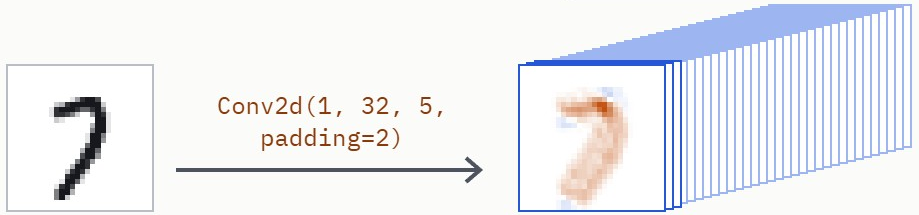
Input channels	1 from the stream
Output channels	32
Kernel	5
Stride	1
Padding	2

```
class ResCNNExpanded(nn.Module):
    def __init__(self):
        super().__init__()
        self.stem = nn.Conv2d(1, 32, 5, padding=2)
        self.b1_norm1 = nn.GroupNorm(1, 32)
        self.b1_conv1 = nn.Conv2d(32, 32, 3, padding=1)
        self.b1_norm2 = nn.GroupNorm(1, 32)
        self.b1_conv2 = nn.Conv2d(32, 32, 3, padding=1)
        self.b2_norm1 = nn.GroupNorm(1, 32)
        self.b2_conv1 = nn.Conv2d(32, 32, 3, padding=1)
        self.b2_norm2 = nn.GroupNorm(1, 32)
        self.b2_conv2 = nn.Conv2d(32, 32, 3, padding=1)
        self.b3_norm1 = nn.GroupNorm(1, 32)
        self.b3_conv1 = nn.Conv2d(32, 32, 3, padding=1)
        self.b3_norm2 = nn.GroupNorm(1, 32)
        self.b3_conv2 = nn.Conv2d(32, 32, 3, padding=1)
        self.norm = nn.GroupNorm(1, 32)
        self.fc = nn.Linear(32, 10)
```

Output channels = 32 means **32 filters**. Each is one shared local weight matrix, 1 × 5 × 5. Together: W is 32 × 1 × 5 × 5 = 800 weights.

One filter, one channel. Each filter slides over the whole digit and makes one 28 × 28 image. 32 filters stack into a **32 × 28 × 28** output.

Kernel, stride, padding: set once for all 32, so every channel comes out **the same size** and they stack. The next conv's filters are 32 × 3 × 3: they read all 32 channels.



x · 1 × 28 × 28

Output: one channel per filter

e	examples in the batch
32	channels: one per filter, index o
28	height, index i
28	width, index j

shape: e × 32 × 28 × 28

Shared settings keep every channel the **same size**, so they stack.

© 2026 Dr. Jesse Roberts

How big is each output channel?

Count the places the window fits. One row of the image is enough: columns work the same way.

OUR NUMBERS

Our first conv, Conv2d(1, 32, 5, padding=2): ⌊(28 + 4 − 5) / 1⌋ + 1 = **28**. Same size in, same size out.

The two-filter slide, 5 × 5, no padding, stride 2: ⌊(28 − 5) / 2⌋ + 1 = **12**.

Every filter in a layer shares N, k, p and s, so **every channel gets the same size.**

The window can slide N − k cells past its first placement: N − k moves, plus the placement it started from.

$$\text{out} = N - k + 1$$

7 − 3 = 4 moves, + 1 = 5

Padding adds p cells on each side:

$$\text{out} = N + 2p - k + 1$$

7 + 2 − 3 = 6 moves, + 1 = 7: same size as the input

Stride s keeps every s-th start; the last partial step is dropped:

$$\text{out} = \lfloor (N + 2p - k) / s \rfloor + 1$$

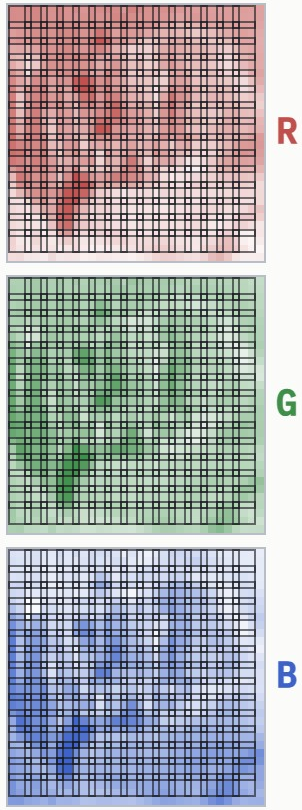
⌊6 / 2⌋ = 3 moves of 2, + 1 = 4

Same settings for every filter means **same-size channels** that stack.

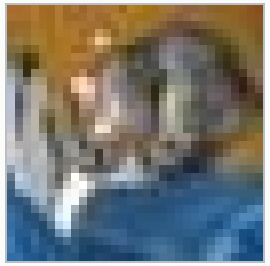
© 2026 Dr. Jesse Roberts

Channels must be the same size

A colour image is 3 channels, R, G and B. One 3 × 3 × 3 filter looks through all three at once.



CIFAR-10 test image



= 3 sheets of 32 × 32, one per channel

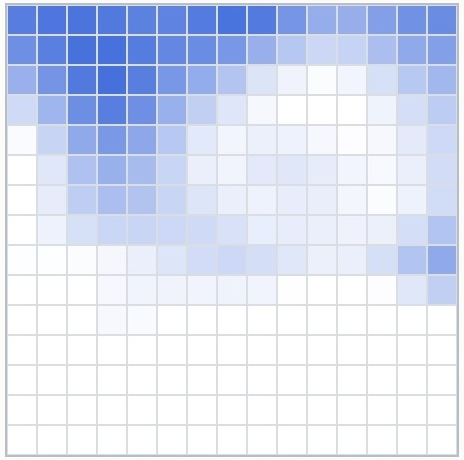
STRONGEST: NEURON 34 OF 225 · WINDOW ROW 2, COLUMN 3

	27 weights			×	pixels		
R	+.3	+.3	+.3	×	.7	.7	.7
	+.3	+.3	+.3		.7	.7	.6
	+.3	+.3	+.3		.7	.7	.7
G	0	0	0	×	.5	.5	.5
	0	0	0		.5	.5	.4
	0	0	0		.5	.5	.5
B	−.3	−.3	−.3	×	.1	.1	.2
	−.3	−.3	−.3		.1	.2	.1
	−.3	−.3	−.3		.2	.2	.2

Σ all 27 → ReLU → **1.36**

This filter: **+0.3 on R, 0 on G, −0.3 on B**, a "warm minus cool" detector. Stride 2.

output: ONE image (15 × 15)



Bright where the image is warm (the orange wall), dark on the blue blanket.

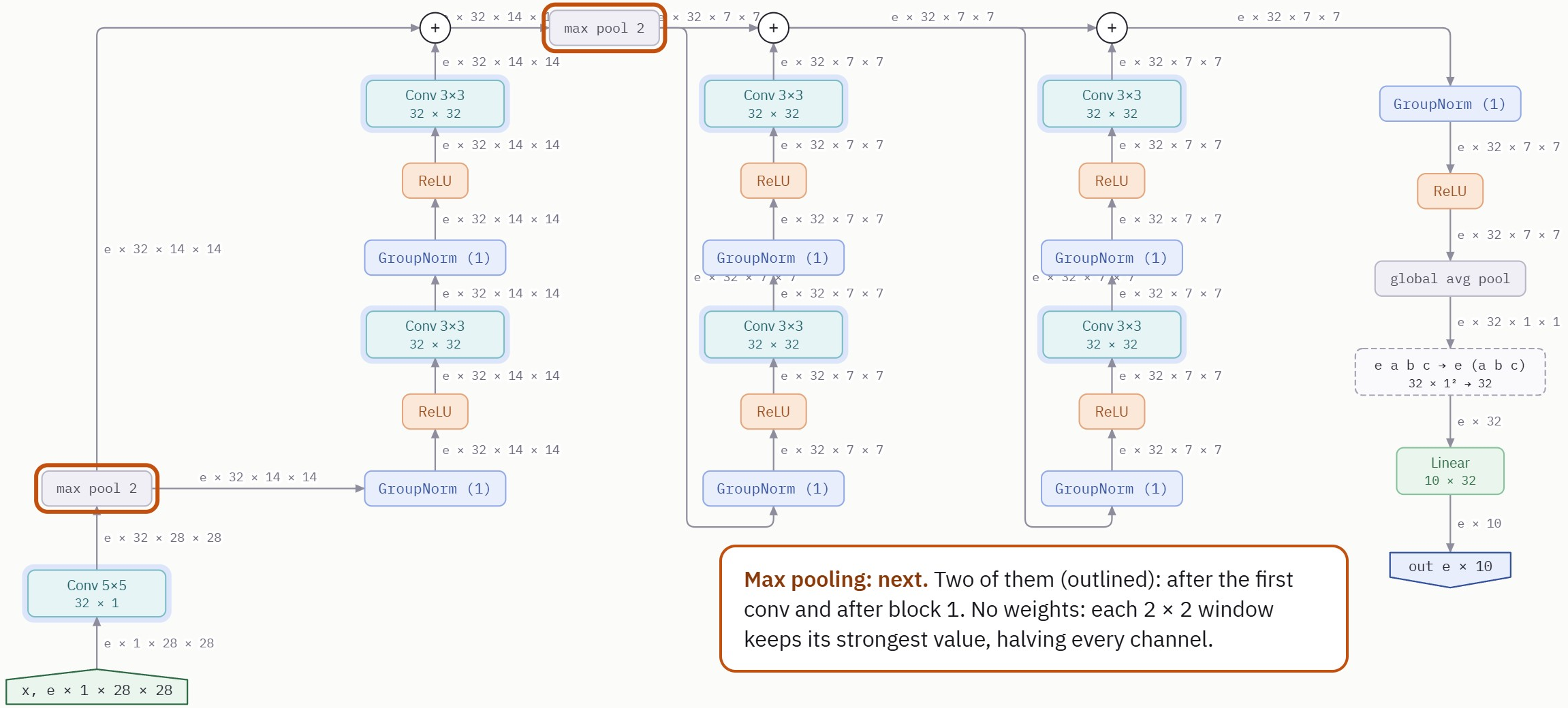
One filter, one image. 225 neurons share these 27 weights, and each sums over all 3 channels, so 3 sheets in give **one** image out. **More filters give more** images or **channels**.

A filter spans **every input channel whether input channels or output from a layer.**
One filter gives one output channel.

© 2026 Dr. Jesse Roberts

Where we are in the network

The residual CNN, read from x at bottom left



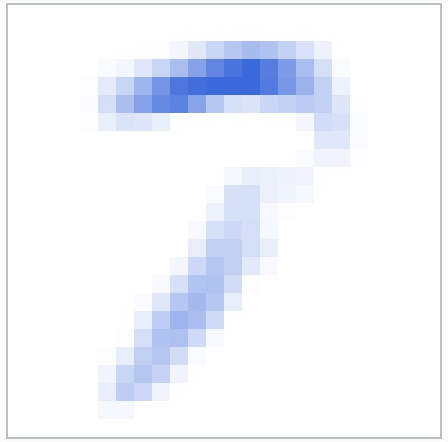
Max pooling: next. Two of them (outlined): after the first conv and after block 1. No weights: each 2 × 2 window keeps its strongest value, halving every channel.

© 2026 Dr. Jesse Roberts

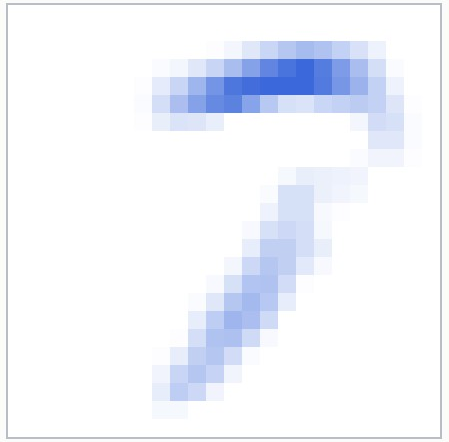
Feature maps still have addresses

The top-edge filter W_1 on the 7, and on the same 7 shifted 3 px right

feature map · original 7

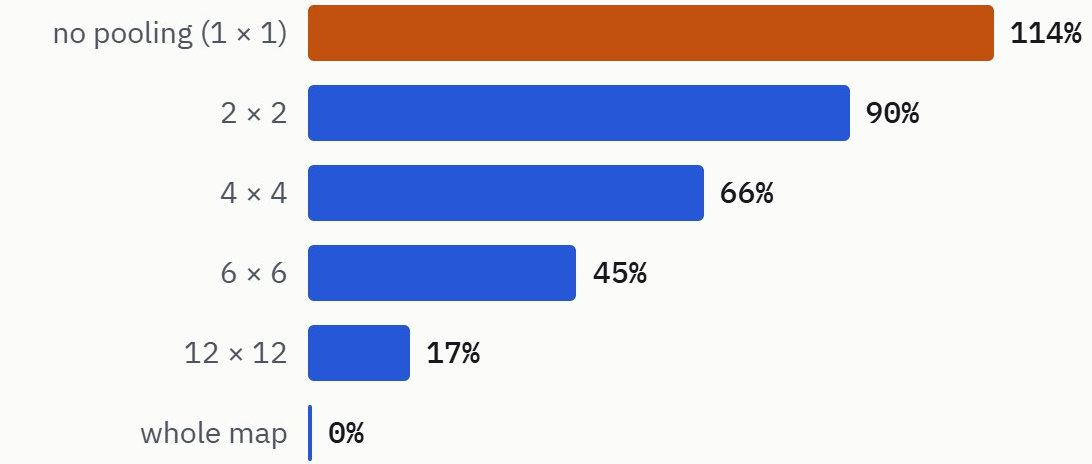


feature map · shifted 3 px



Shift in, shift out: the same top edges, 3 columns over. **207 of 576 cells** changed value.

Keep only the strongest value in each region: how much does the result change after the shift?



Enter max pooling. "A top edge was found somewhere in this 2 × 2 region." Small regions, repeated after convs, forgive position a little at a time; the global average pool at the end forgives the rest.

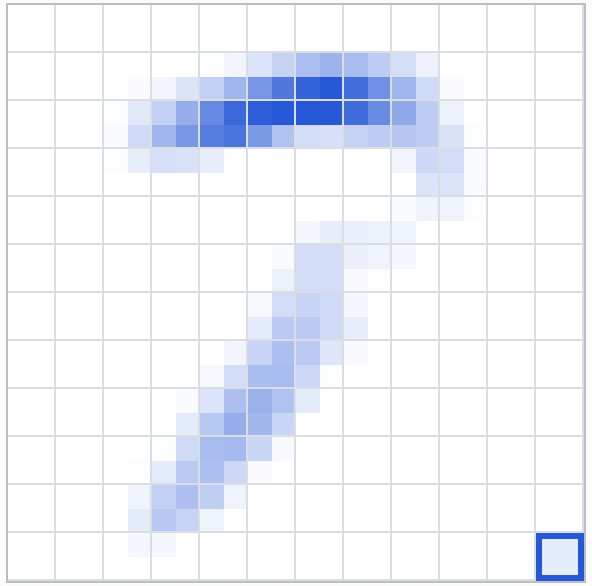
Max pooling keeps **what** was found and blurs **exactly where**.

© 2026 Dr. Jesse Roberts

Max pooling keeps each local window's winner

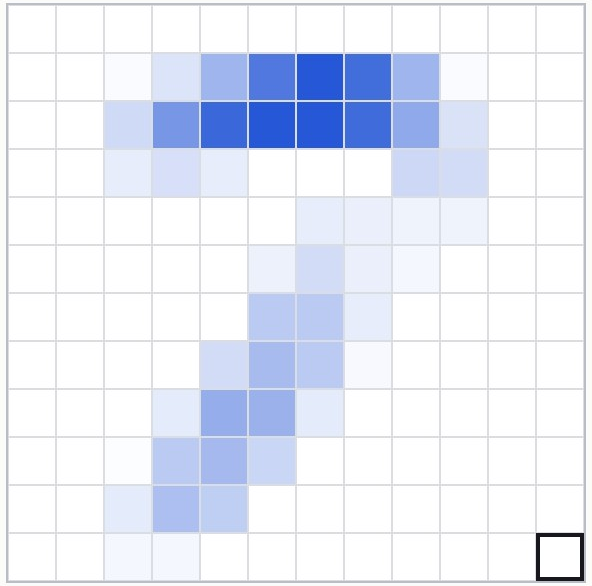
The top-edge map of the 7 after ReLU. Each 2 × 2 window lets only its largest value through.

ReLU(ŷ) (24 × 24)

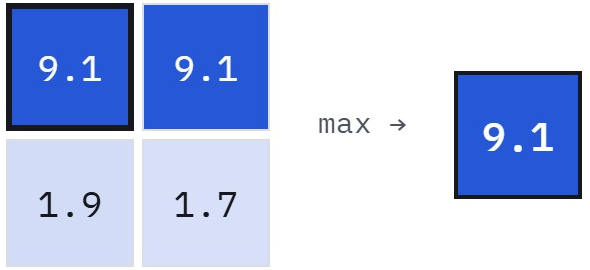


max, 2 × 2

pooled (12 × 12)



STRONGEST WINDOW: ROW 2, COLUMN 6



4 values in, 1 out: the largest. The other 3, and which corner it sat in, are dropped.

A gate, not a layer of neurons. $\text{out} = \Sigma_i\, g_i\, v_i$, $g = \text{one-hot}(\text{argmax } v)$: each value scores itself and only the winner passes. No weights; the gradient flows only to the winner.

Stride 2 is what shrinks the map, 576 values to 144. At stride 1 it would stay 24 × 24.

Keep this picture: make the gate soft and learned, and it becomes **attention**.

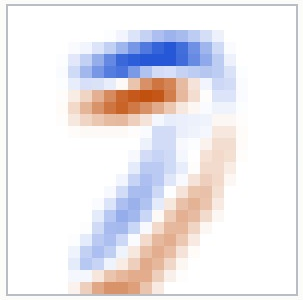
Max pooling keeps **whether** a feature fired in each patch, not exactly **where**.

© 2026 Dr. Jesse Roberts

From equivariant to invariant: pooling

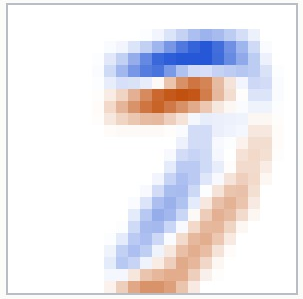
A feature map still says where. The label only asks what, so the network has to throw the where away.

Global average pooling



→ `mean(ReLU(·))` → **0.610**

One number per filter: **how much top edge** the image has, not where.

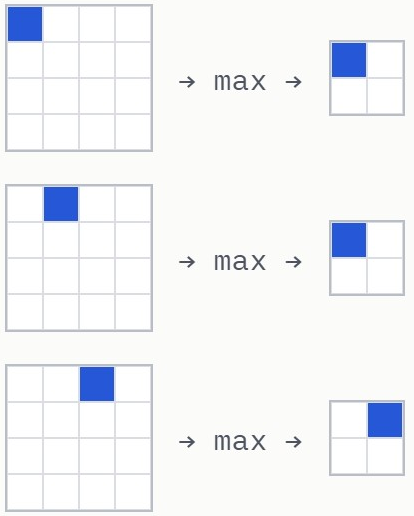


→ `mean(ReLU(·))` → **0.610**

Identical. Any shift that keeps the 7 on the canvas gives the same number.

`top: original 7 · bottom: shifted 3 px`

Max pooling, 2 × 2 windows, stride 2



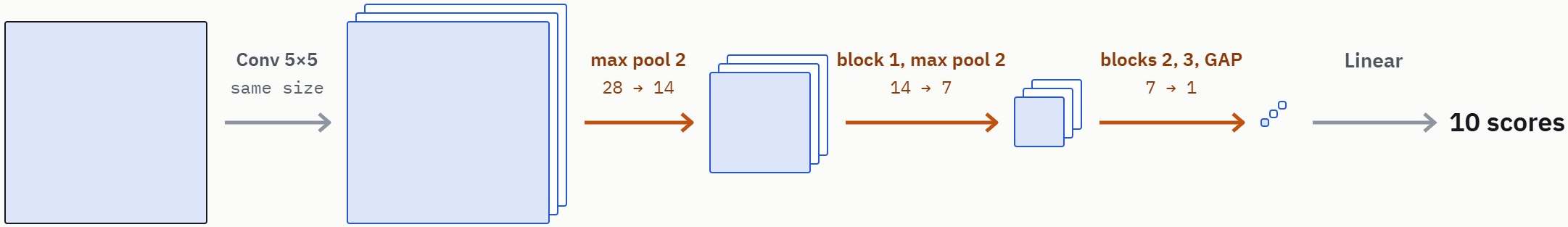
A strong response anywhere in a window gives the same output: **invariant to small shifts**, equivariant at the coarser grid.

Convolution is **equivariant**; pooling makes the answer **invariant**.

© 2026 Dr. Jesse Roberts

The network shrinks where, keeps what

Follow the shapes passed along our network: the channels stay, the image gets smaller



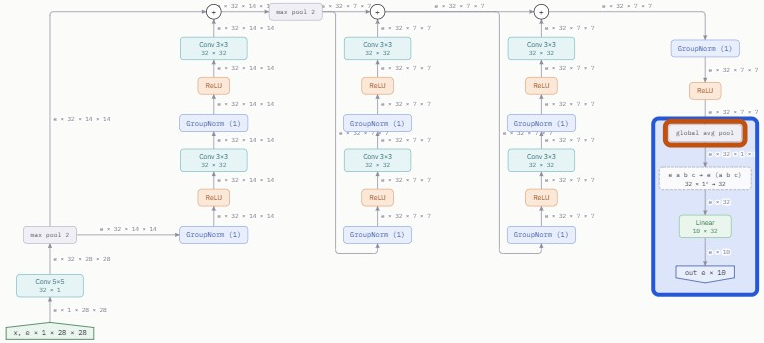
`x · 1 × 28 × 28`
784 positions

`e × 32 × 28 × 28`
784 per channel

`e × 32 × 14 × 14`
196 per channel

`e × 32 × 7 × 7`
49 per channel

`e × 32 × 1 × 1`
1 per channel



GAP finishes the job. 7 × 7 → 1 × 1: one number per channel, "how much of this feature", no position at all. The Linear layer reads 32 numbers.

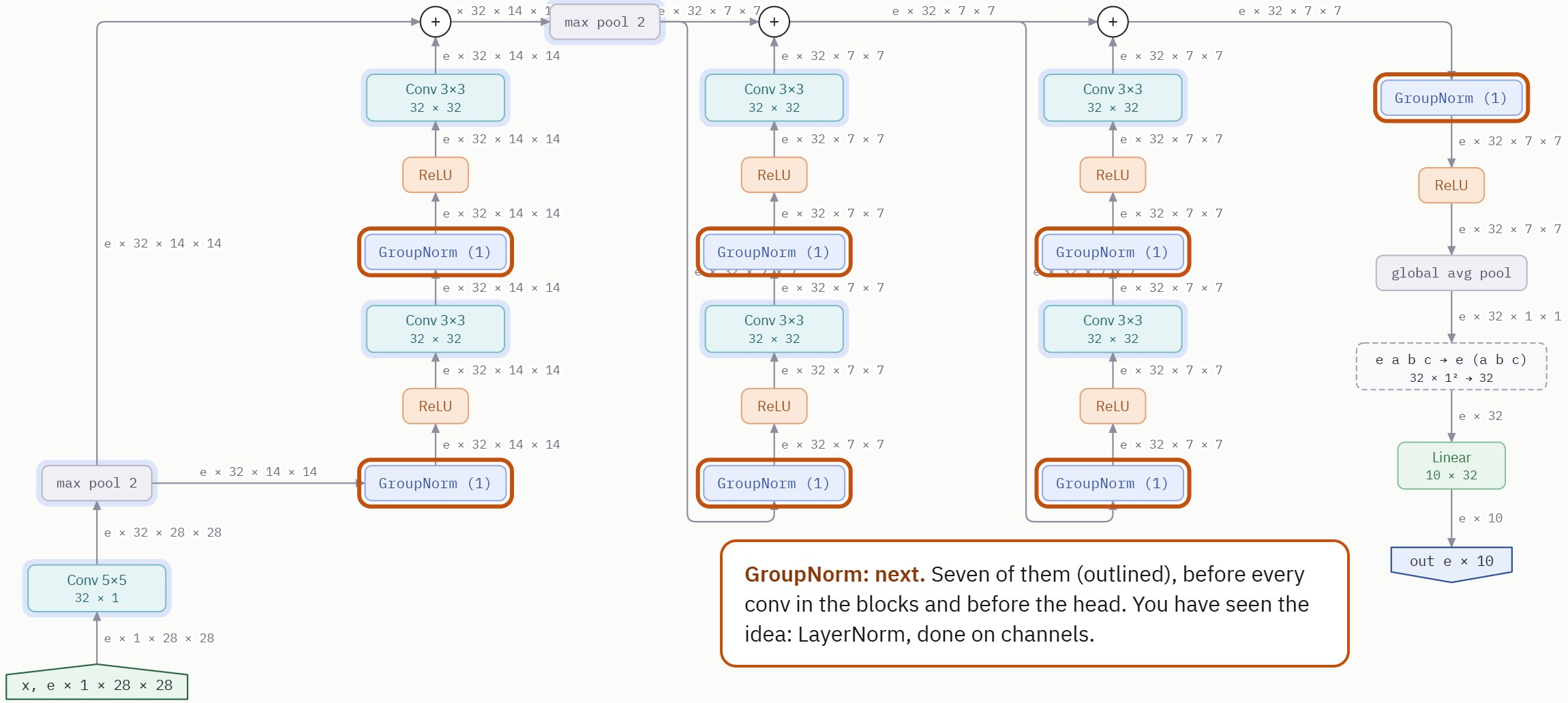
`shaded: the layers working at this size · orange: what shrank it`

Max pool and GAP shrink the image until **only what is left**.

© 2026 Dr. Jesse Roberts

Where we are in the network

The residual CNN, read from x at bottom left



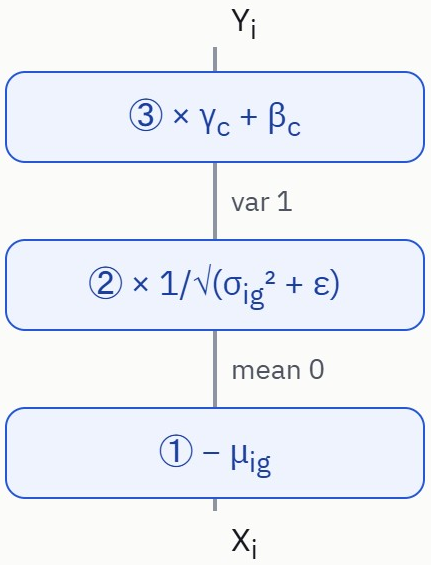
GroupNorm: next. Seven of them (outlined), before every conv in the blocks and before the head. You have seen the idea: LayerNorm, done on channels.

© 2026 Dr. Jesse Roberts

GroupNorm: LayerNorm over groups of channels

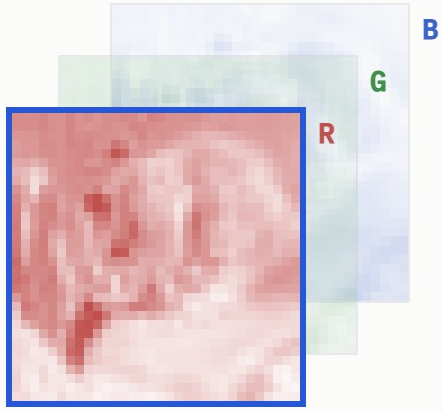
One example at a time: the groups decide which part of the example's row gives μ and σ²

The norm, per example i, group g



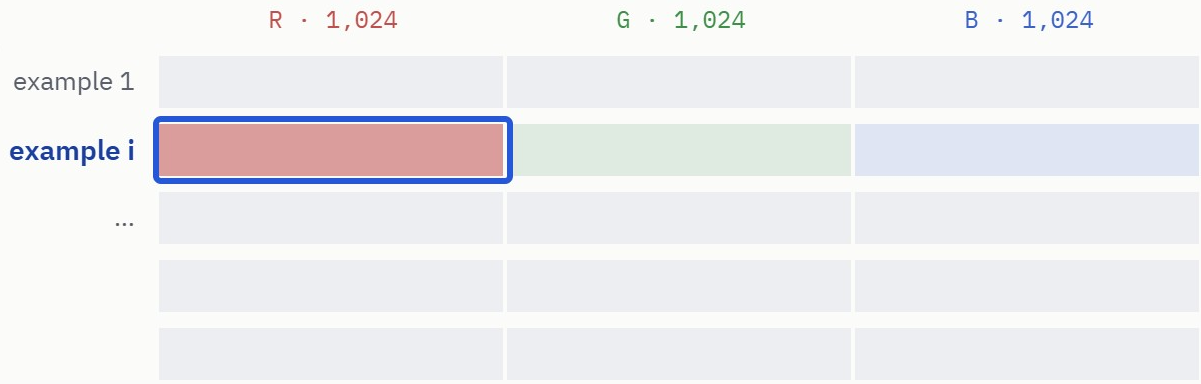
μ_{ig}, σ_{ig}^2: mean and variance of the values in group g of example i. γ, β: one per channel.

ONE EXAMPLE · 3 × 32 × 32



a CIFAR-10 cat as its R, G and B channels

THE BATCH · EXAMPLES × FEATURES



Each row is one example, flattened channel by channel: 3 × 32 × 32 = 3,072 features.

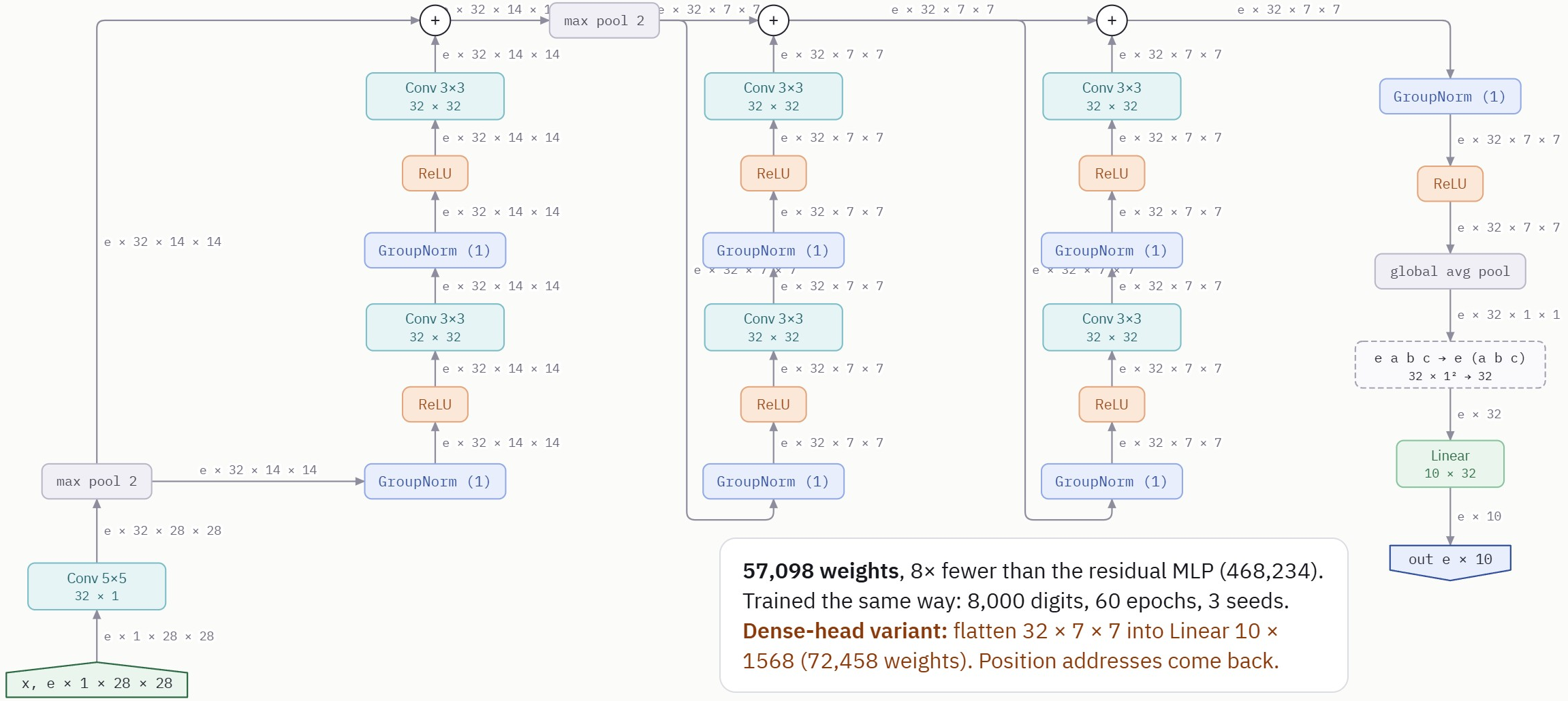
It can't. Every group holds C / G whole channels, so G must divide C: PyTorch rejects GroupNorm(2, 3) with an error. Our layers have 32 channels, so G can be 1, 2, 4, 8, 16 or 32; we use 1.

G must divide C: every group holds **C / G whole channels**.

© 2026 Dr. Jesse Roberts

The whole network

Start at x, bottom left: three residual blocks, then the head. torch-canvas, every layer drawn.



57,098 weights, 8× fewer than the residual MLP (468,234). Trained the same way: 8,000 digits, 60 epochs, 3 seeds. **Dense-head variant:** flatten 32 × 7 × 7 into Linear 10 × 1568 (72,458 weights). Position addresses come back.

© 2026 Dr. Jesse Roberts

Why is the network built this way?

Answer each one with a neighbour before the reveal

QUESTION 1

Why not a max pool after every block?

What do back-to-back residual blocks at 7 × 7 buy us?

QUESTION 2

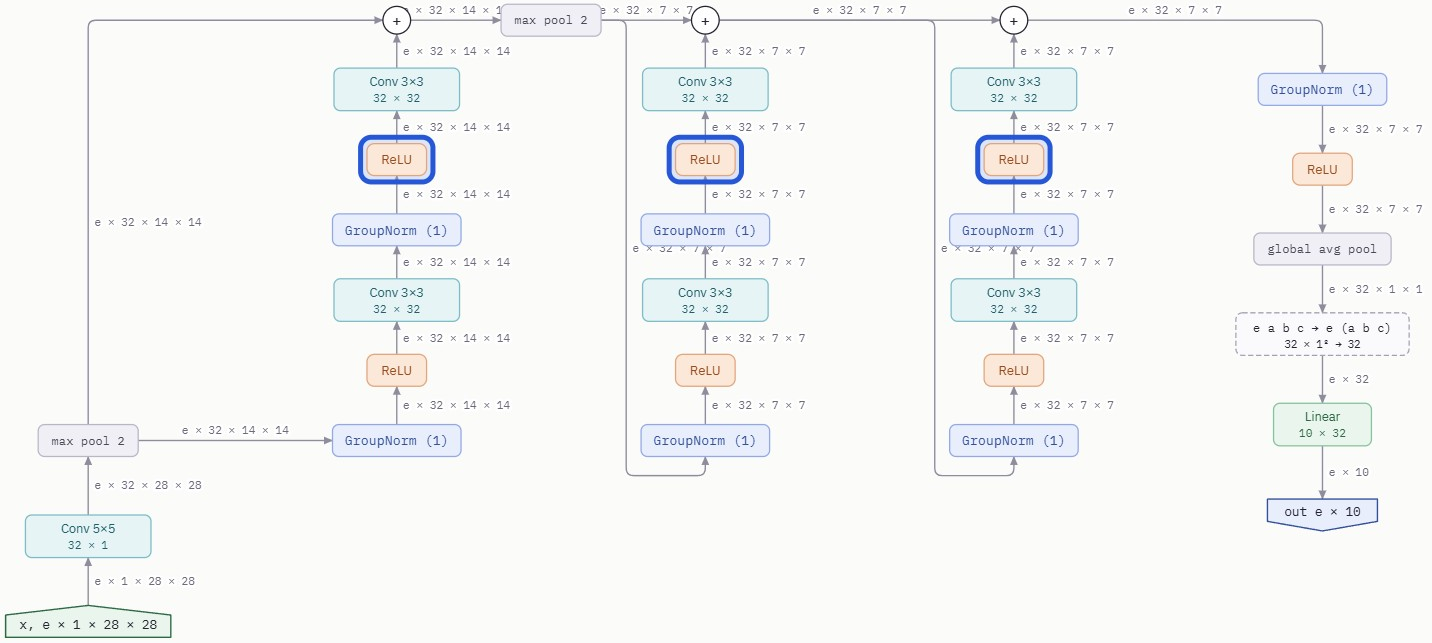
Why a GroupNorm inside every block?

Before each conv, not just once at the start.

QUESTION 3

Why a ReLU between the two convs?

What would a block compute without it?

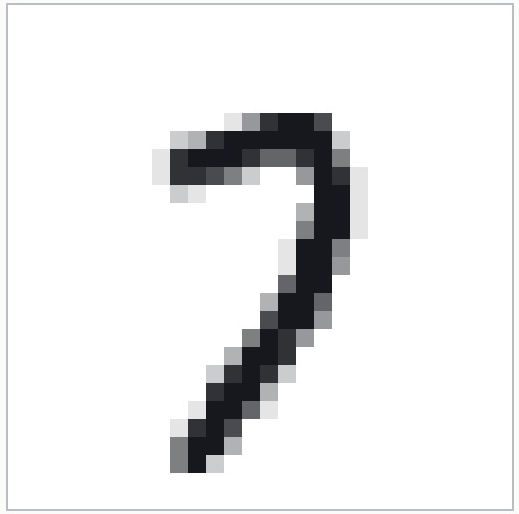


ANSWER 3 · Two convs with nothing between them are one linear map: a 3 × 3 after a 3 × 3 is just a 5 × 5. The activation is what lets a block compute something new. Remove every activation and, apart from pooling, the whole network is one big linear filter.

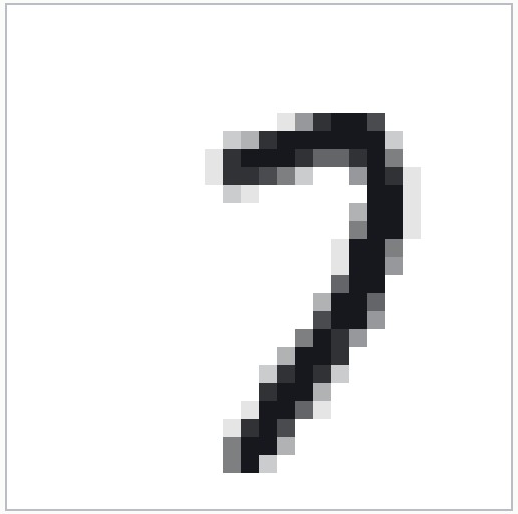
© 2026 Dr. Jesse Roberts

Will the CNN pass the shift test?

Trained on the same 8,000 centered digits as Thursday's MLP. No augmentation.



test digit, as is



shifted 3 px right

Thursday's MLP on these shifted digits: **45%**.

Every test digit moves 3 px right. What will the CNN with the GAP head score?

A **~95%**

B **~80%**

C **~50%**

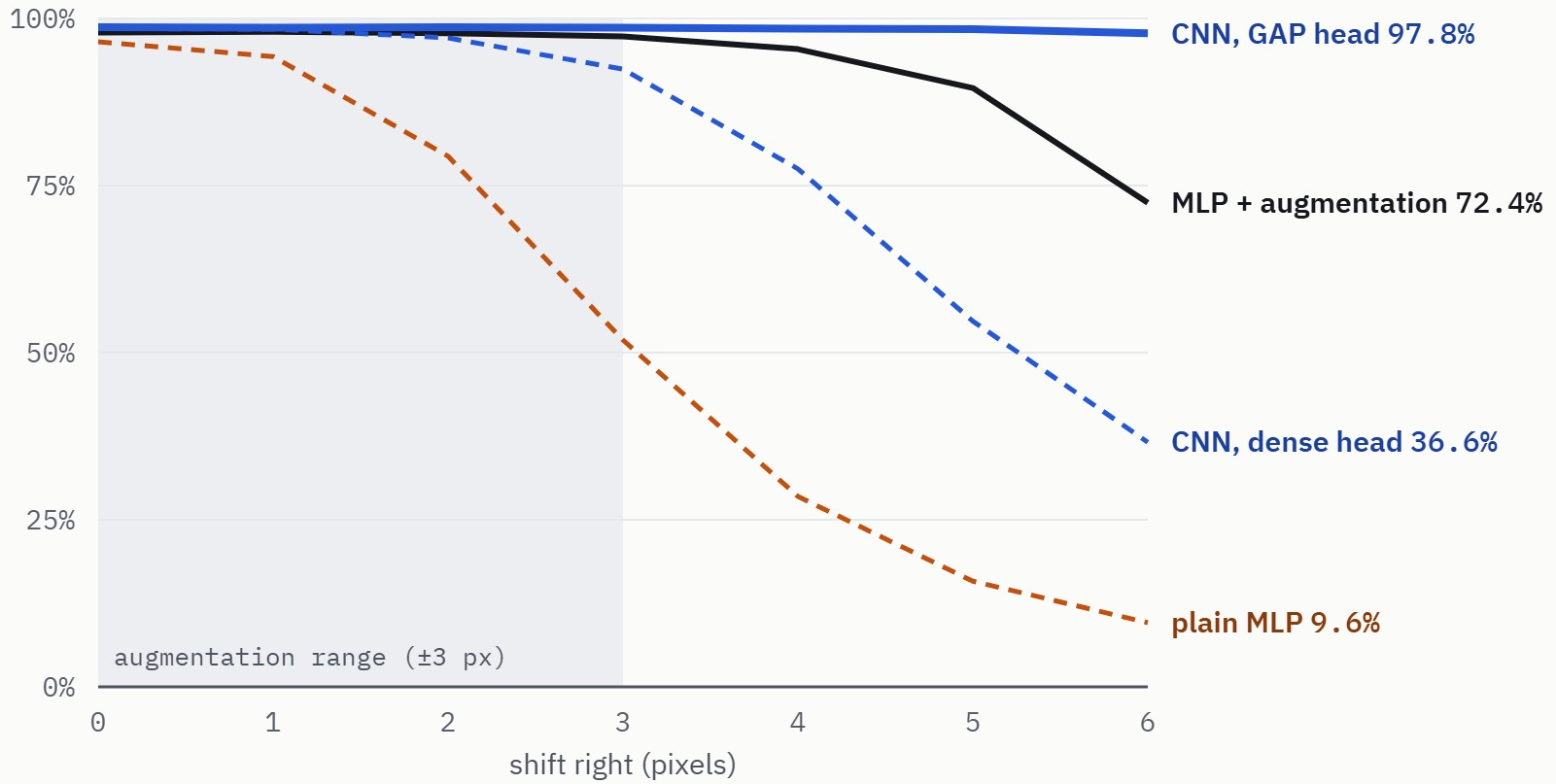
D **~10%**

Then: will the **dense head** do better or worse? Why?

© 2026 Dr. Jesse Roberts

The shift test, rerun

Accuracy on the 2,000 test digits shifted right, after training on 8,000 centered digits



Mean of 3 seeds. Residual MLPs with LayerNorm, rerun on today's split (Thursday: 95.6%, 45%, 8% at 0, 3, 6 px).

AT 6 PX, NEVER SEEN BY ANY MODEL

MLP + augmentation

72.4%

CNN, GAP head, no augmentation

97.8%

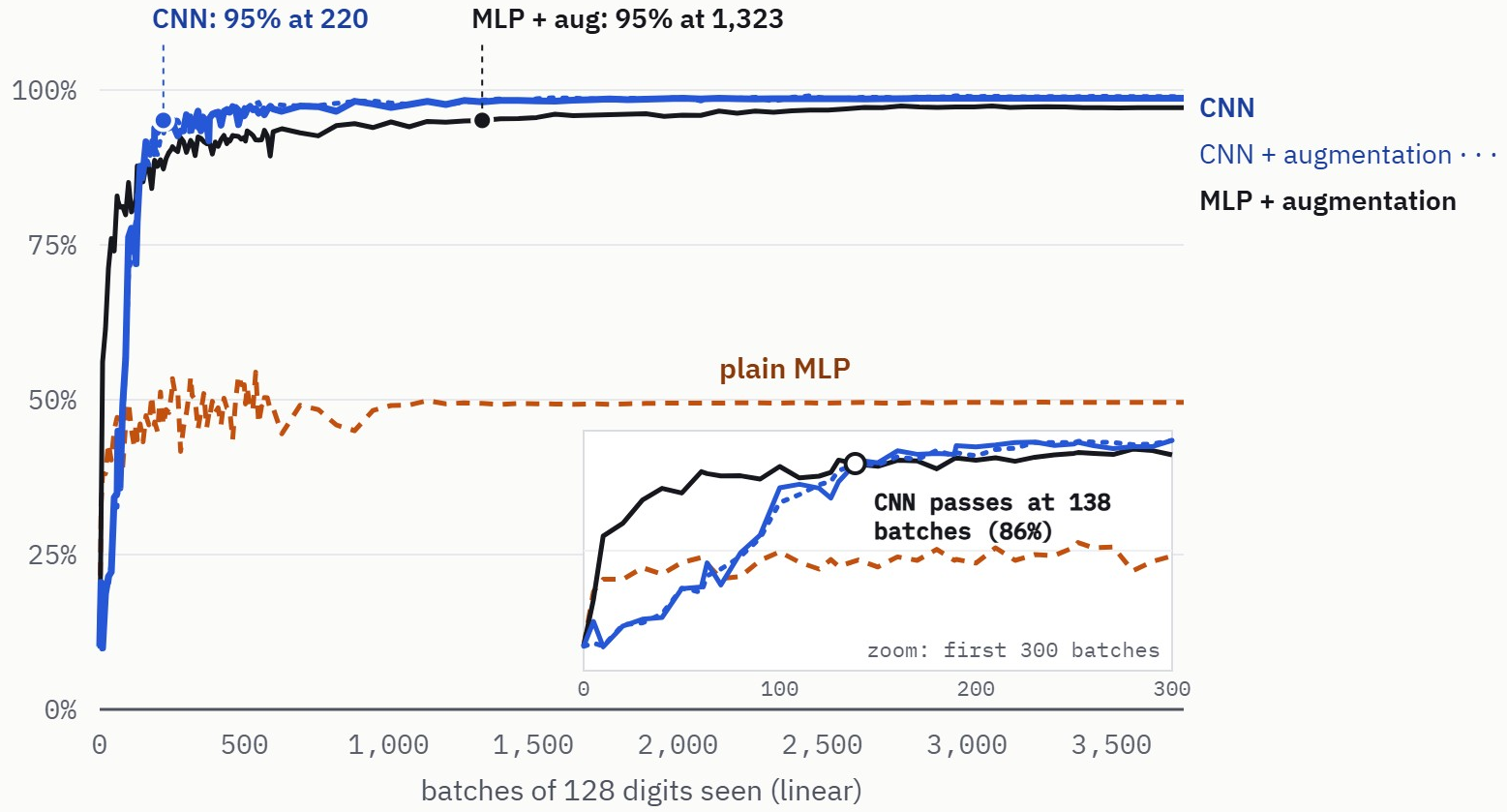
Even inside ±3 px the CNN is ahead (98.6% vs 97.3% at 3 px). Past it, only the built-in bias holds.

Taught invariance fades past what was shown; built-in invariance **holds at any shift**.

© 2026 Dr. Jesse Roberts

Bias is worth data

Accuracy on test digits shifted 3 px right, against the number of training batches each network has seen



BATCHES TO 95%

CNN: 220

MLP + augmentation: 1,323

Plain MLP: never (best 54.4%)

THE CROSSOVER

Augmentation gets there first: 80% after **60** batches, the CNN after **140**. The CNN catches up at about **138** batches (86%) and stays ahead; the augmented MLP crawls from there.

Every network: batches of 128 from the same 8,000 digits, so one batch is the same 128 gradient estimates. 1 seed, scored every 10 batches up to 600, every 63 after.

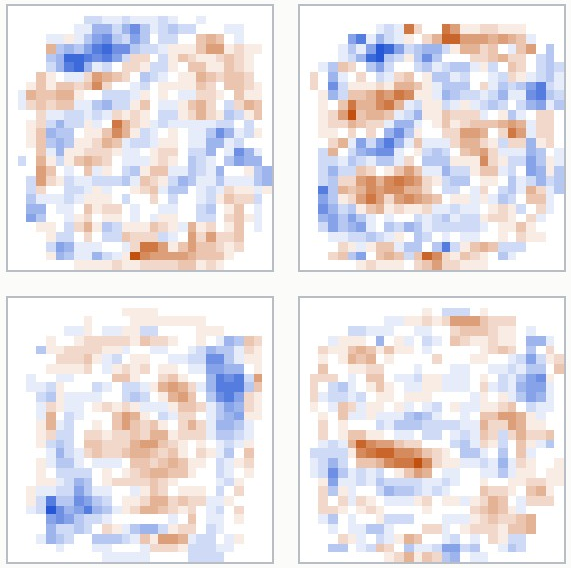
Augmentation starts faster; the matching bias **keeps climbing** and reaches 95% in 6× fewer batches.

© 2026 Dr. Jesse Roberts

What each model learned

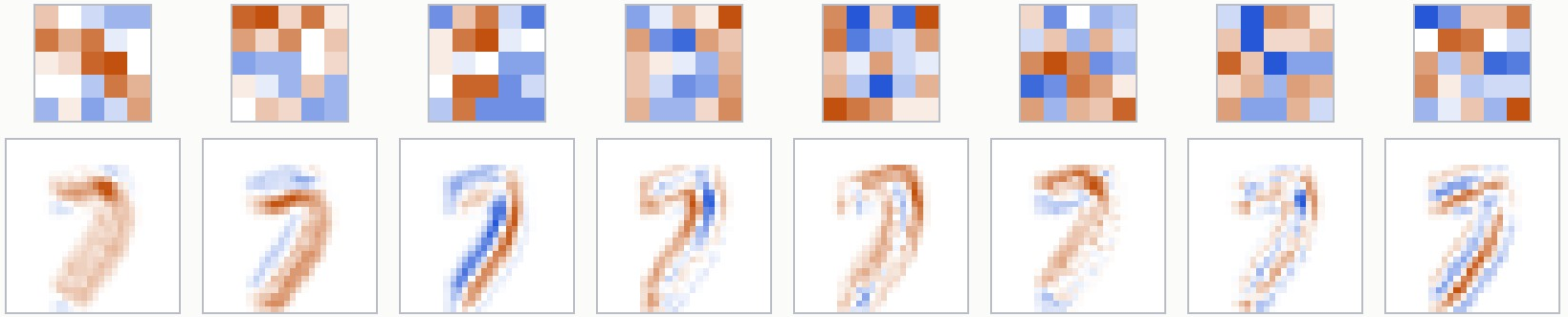
Thursday asked what the MLP actually learned. Here are both models' first layers side by side.

MLP: four first-layer units



784 weights per unit, each tied to one pixel. It learned **where ink usually is**.

CNN: eight of its 32 first-layer 5 × 5 filters, and what each finds in the 7



top: a learned filter W · bottom: its response everywhere on the 7

positive weight or response negative near 0

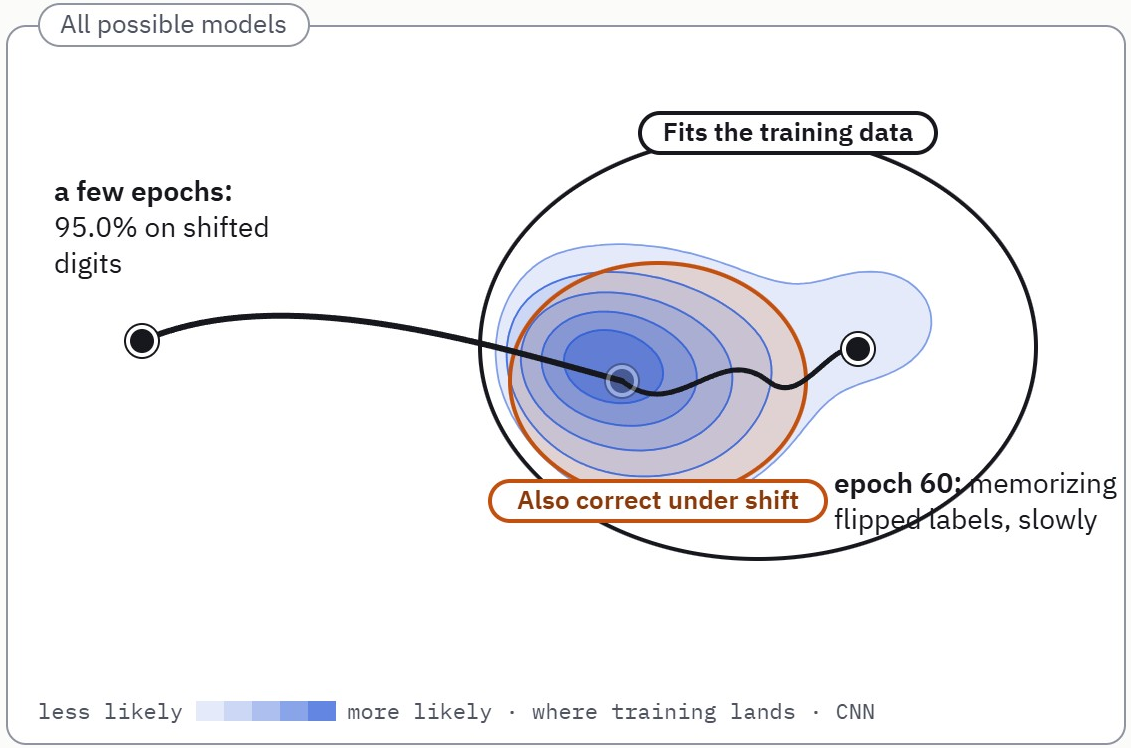
25 weights per filter, used at every position. Each one picks out a kind of edge or stroke, and its map lights up **wherever that stroke is**.

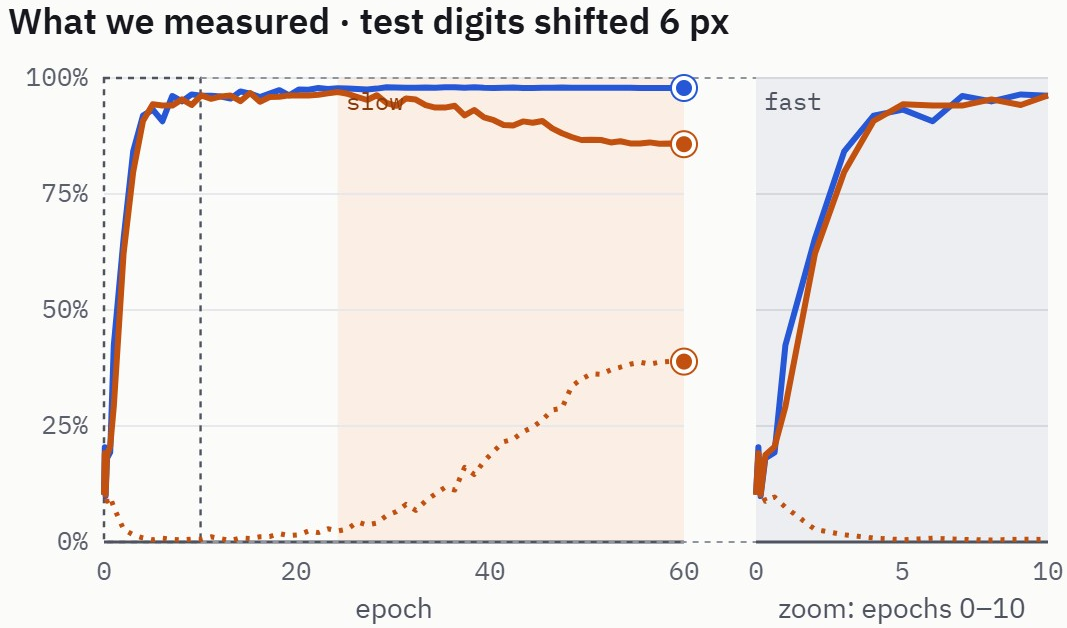
The CNN learned **what a stroke looks like**; the MLP learned where ink usually is.

© 2026 Dr. Jesse Roberts

The CNN takes the same road

The residual CNN on the same 8,000 digits, now scored on test digits shifted 6 px





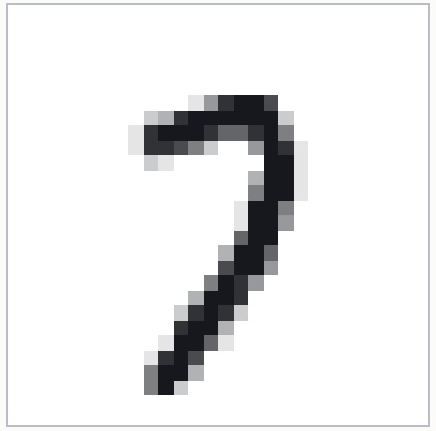
True labels: rises with clean accuracy and stays (97.9%). Flipped labels: peaks at 97.0% (epoch 24), falls to 85.8%. It memorizes **more slowly than the MLP**: 39% of flipped labels by epoch 60, not 100%.

The architecture picks **where** SGD lands first; it does not stop the slow drift into **memorizing**.

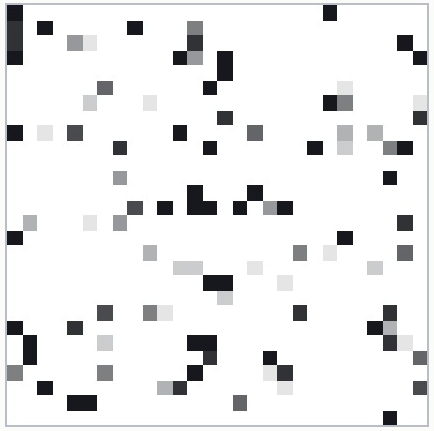
© 2026 Dr. Jesse Roberts

Can a local model learn a global pattern?

Talk it over with a neighbour before we move on



original



same fixed shuffle for every image

MLP, normal pixels

95.6%

MLP, shuffled pixels

94.7%

Thursday: retrained from scratch on shuffled images, the MLP did not care.

WE LEFT THIS OPEN

Can a CNN learn global patterns?

Now train the CNN from scratch on the same shuffled images. It only ever looks through 5 × 5 and 3 × 3 windows. Its accuracy will be…

A

about the same, like the MLP

B

a few points lower

C

near chance (10%): nothing local to learn

Hint: what does a 5 × 5 patch of a shuffled image contain?

© 2026 Dr. Jesse Roberts

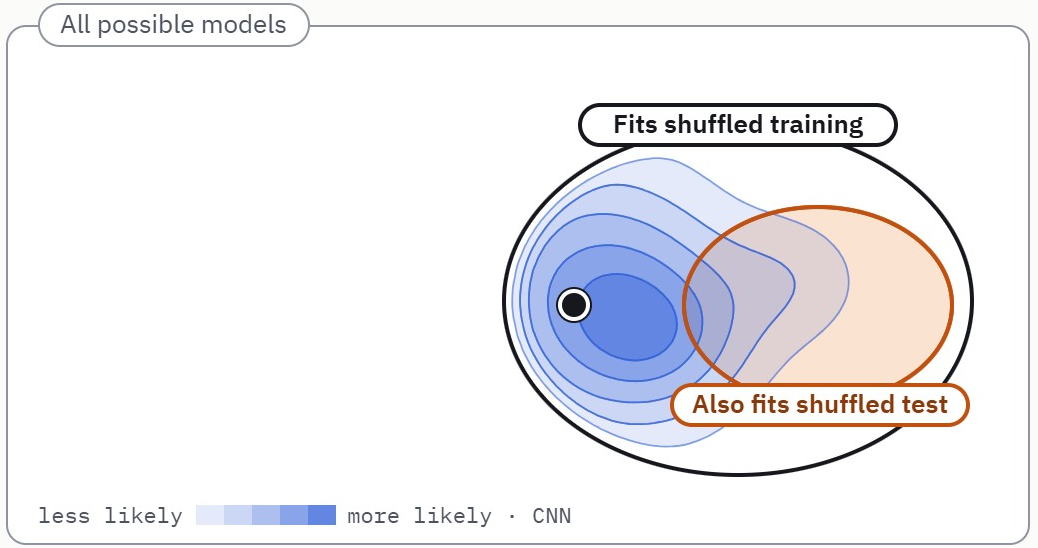
When the assumption is wrong

Trained from scratch and tested on the same fixed pixel shuffle; 8,000 digits, 60 epochs

	normal pixels	shuffled pixels
MLP Thursday	95.6%	94.7%
CNN GAP head	98.7%	91.7%

B. Far above chance (10%): a model that only looks locally **still learns a pattern spread over the whole image**. But it loses 7 points while fitting every shuffled training digit (100% train): a 5 × 5 patch holds 25 unrelated pixels, no strokes to find.

CNN: mean of 3 seeds, shuffled 91.4–92.0%. MLP numbers from Thursday.



On shuffled data the CNN **rarely lands** among the models that also pass the test. Training still finds a fit, just seldom one of those.

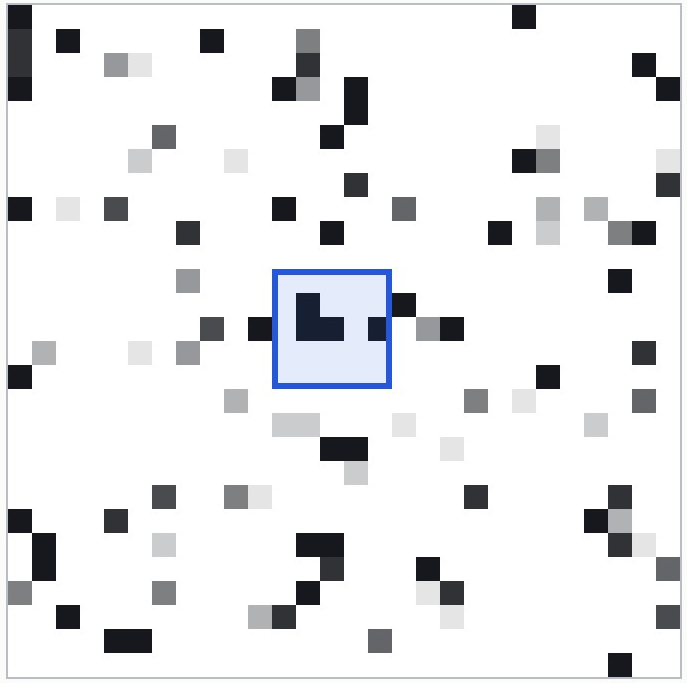
STRENGTH OR WEAKNESS? Both. Ignoring pixel order costs the MLP nothing when order carries no information, and costs it data when order does.

A bias is a bet: it **pays when it is true and costs when it is false**.

© 2026 Dr. Jesse Roberts

How does it see the whole digit?

Talk it over with a neighbour before we move on



the shuffled 7 · one 5 × 5 filter window

THE PUZZLE

Every filter in our CNN covers **5 × 5** or **3 × 3** pixels. On shuffled digits it still gets **91.7%**, though the pixels that matter can sit 20 px apart.

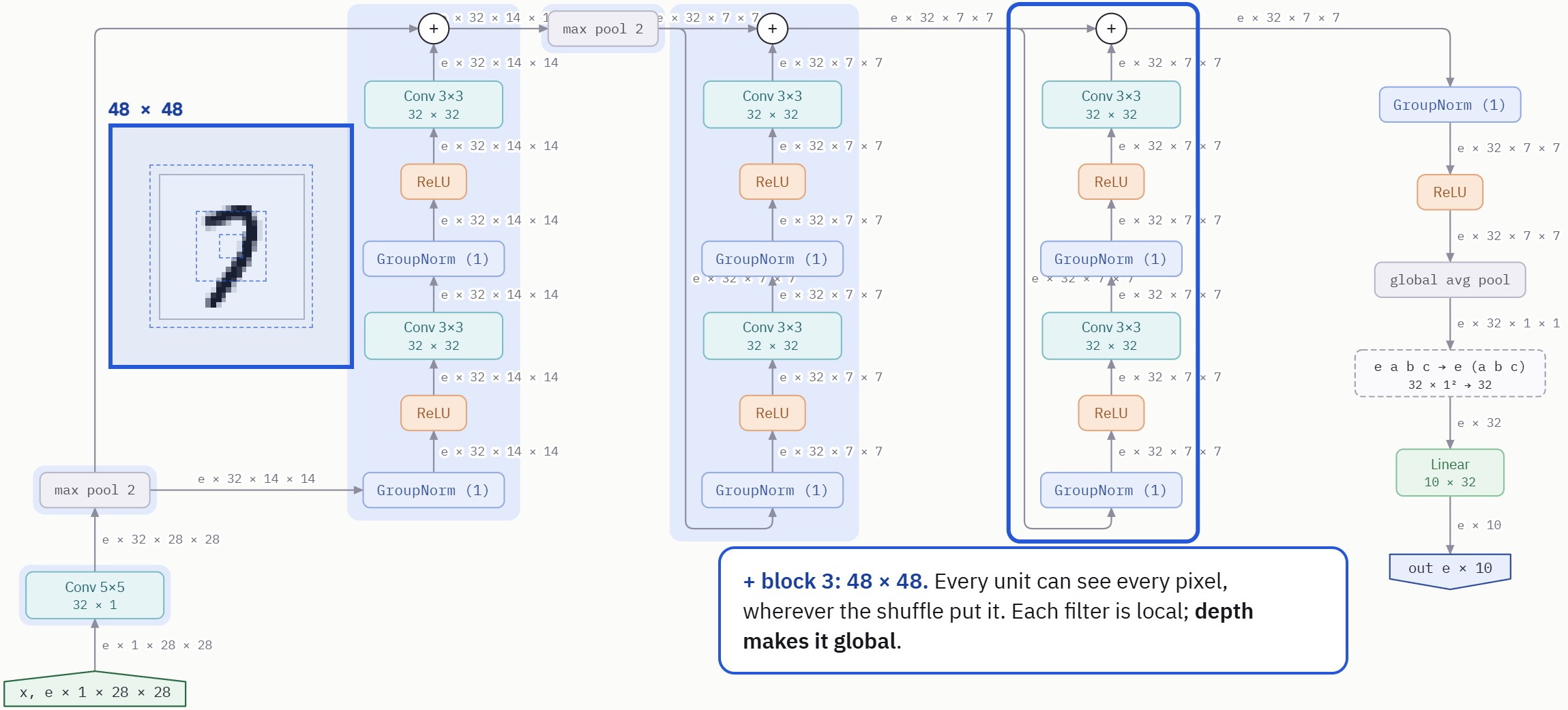
How can a model that only looks locally use them together?

Hint: pick one unit in the last residual block and trace it back. Which input pixels can change its value?

© 2026 Dr. Jesse Roberts

Receptive field grows in depth

The **receptive field**: the area of the original input that can impact a centered filter's output at a layer.

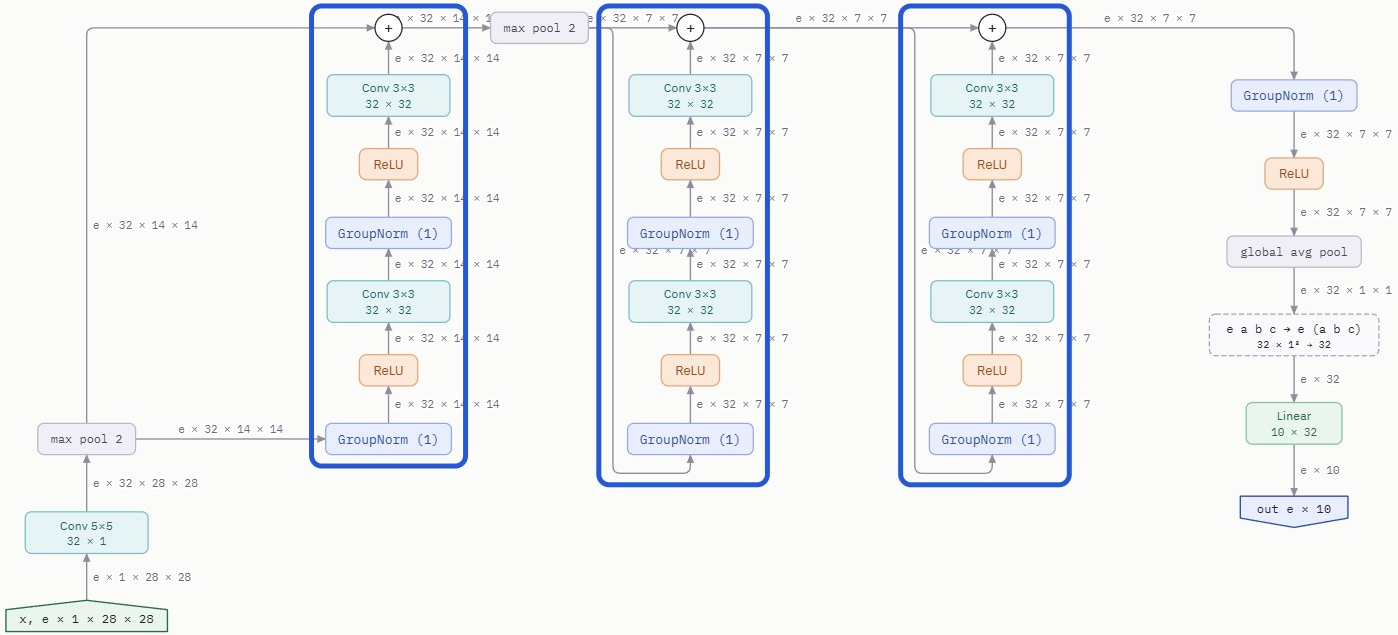


+ block 3: 48 × 48. Every unit can see every pixel, wherever the shuffle put it. Each filter is local; **depth makes it global**.

© 2026 Dr. Jesse Roberts

Depth makes it global

Cut the network short and the field shrinks. Test accuracy, 1 seed; fewer blocks also means fewer weights.



blocks kept	0	1	2	3
field of last layer	6^2	14^2	32^2	48^2
normal digits, %	64.4	97.6	98.7	98.8
shuffled digits, %	67.3	90.9	92.4	91.4

3 blocks, the full network: 48 × 48. Normal digits barely needed depth; shuffled ones needed it: 67.3% → 91.4%.

Universal again. Deep enough, a CNN can approximate any continuous function, just like the MLP.

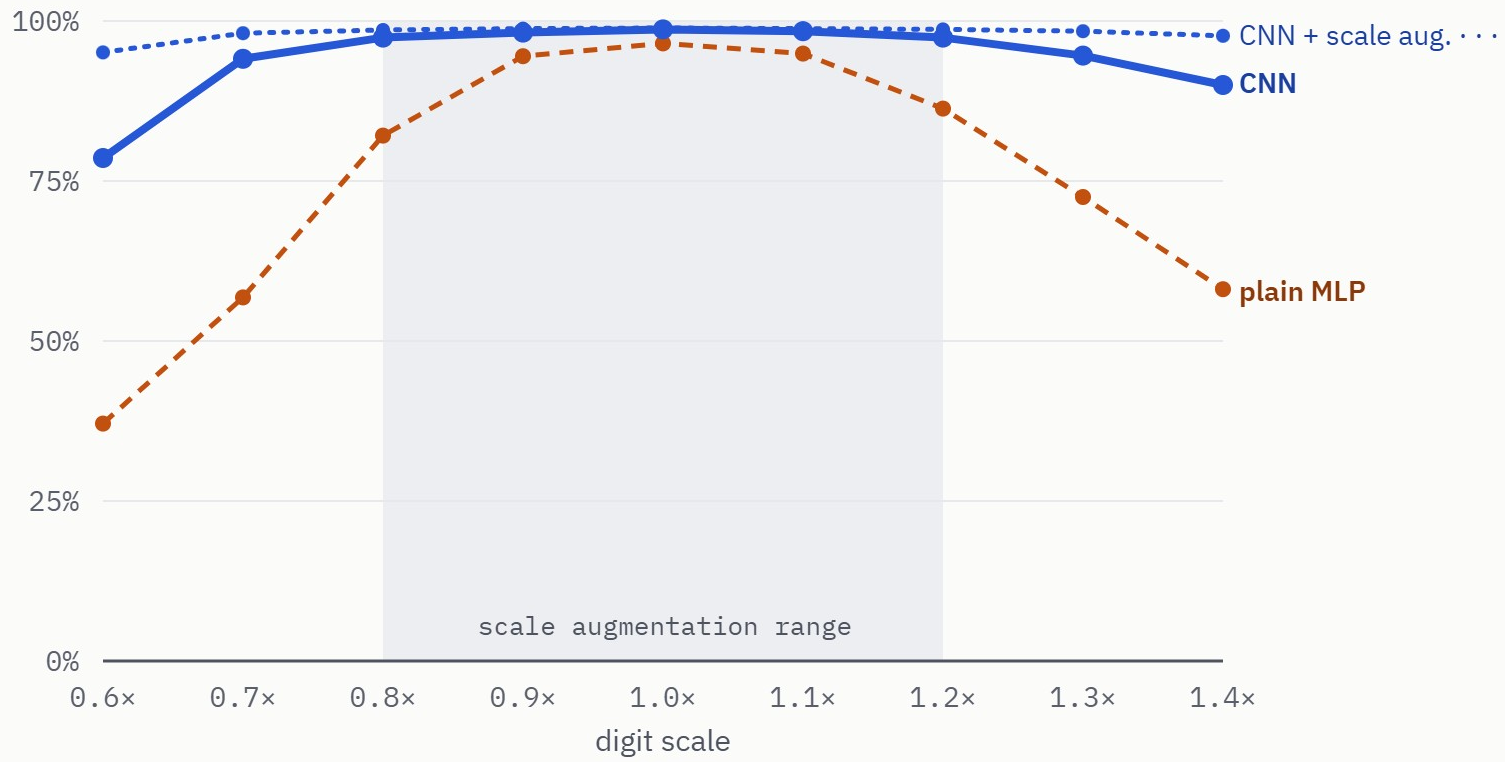
Zhou, Appl. Comput. Harmon. Anal. 2020

Locality limits what one layer sees, **not what the network can learn**.

© 2026 Dr. Jesse Roberts

Only the invariances you build in

Thursday's scale test: test digits rescaled about the center, models trained on 8,000 digits



Bilinear resampling, zero fill. MLP and CNN: mean of 3 seeds; CNN + scale augmentation (0.8–1.2×): 2 seeds.

Not built in, only helped

A slightly bigger stroke still looks like a stroke to a 5 × 5 filter, so the CNN beats the MLP. But at 1.4× it falls to **90%**, at 0.6× to **79%**.

Add the other lever

Same CNN, trained with random rescaling: **98%** at 1.4× and **95%** at 0.6×, with shift still built in.

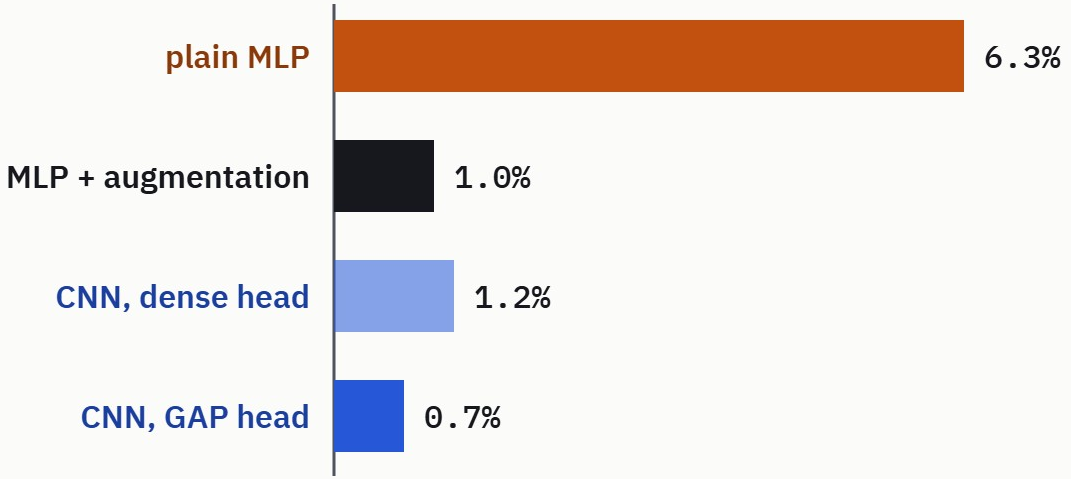
Build in what you can; **augment for the rest**.

© 2026 Dr. Jesse Roberts

Almost invariant

A CNN's invariance comes from details of the architecture, and those details leak

Test digits whose prediction changes after a 1 px shift



2,000 test digits, mean of the 4 directions and 3 seeds.

Far better than the plain MLP, a little better than an MLP shown shifts, but not zero.

Why not 0%? Stride.

Max pooling keeps every other position. A 1 px shift changes which values share a window, so the pooled map is not just shifted: it is different (aliasing).

Azulay & Weiss (JMLR 2019); fix: blur before downsampling, Zhang (ICML 2019)

Borders matter too

A plain CNN without padding sees edge pixels less often than central ones. At 3 px: **87.7%**; at 6 px: **52.7%**. The same CNN padded: 96.3% and 89.5%.

Plain (non-residual) CNN; unpadded result: one seed

Convolution is equivariant by design; the whole network is **approximately invariant** .

© 2026 Dr. Jesse Roberts

Two levers compared

The same knowledge, "a shifted 7 is still a 7", written in two places. Numbers from today's runs.

	Augmentation: change the data	Architecture: change where training lands
Where it lives	In the training data: every batch shows shifted copies.	In the weights: small patches, one filter used everywhere.
How exact	Only the range shown: 97% at 3 px, 72% at 6 px.	Any shift, by construction: 99% at 3 px, 98% at 6 px. Pooling and borders leak a little.
Batches needed	Faster to 80% (60 batches), but 1,323 batches to reach 95% on shifted digits.	140 batches to 80%, then passes it: 220 batches to 95%, 6× fewer.
Cost	More training: 10^8 versions for shift × scale × rotation.	None at training time; 8× fewer weights (57k vs 468k).
Which invariances	Anything you can program: shift, scale, rotation, color.	Only what fits the weights: shift. Others need new designs.
When it is wrong	Flipped digits would teach wrong labels.	Shuffled pixels: 98.7% drops to 91.7%.

Augmentation **shows** the invariance; architecture **assumes** it.

© 2026 Dr. Jesse Roberts

Weaker bias, more data

The same trade-off at scale, on CIFAR-10: whatever is not built in has to be learned from examples

less built in → more built in

MLP

No locality, no sharing.

CIFAR-10 from scratch: **75%** at best, even with augmentation and 5,000 epochs.

After pretraining on ImageNet-21k (about 14M images): **95%**.

About **280 ×** the images the CNN needed to get close.

Bachmann, Anagnostidis & Hofmann (NeurIPS 2023)

CNN

Locality and shift built in.

CIFAR-10 from scratch, only its 50,000 training images: **93.6%** (ResNet-110).

Today: 95% on shifted digits after 220 batches; the MLP with augmentation needed 1,323.

Pays for it in flexibility when the assumption doesn't hold, as the pixel shuffle showed.

He, Zhang, Ren & Sun (CVPR 2016)

Other symmetries, other architectures. Rotations → group-equivariant CNNs (Cohen & Welling 2016). Relabelling a graph's nodes → graph neural networks. Reordering a set → DeepSets. Same recipe: name the symmetry, build it into the weights.

Less bias costs **data**; more bias costs **flexibility**.

© 2026 Dr. Jesse Roberts

Choose the bias your data has

Your project bans convolutions, so augmentation has to carry every invariance your data has

Label survives…	MNIST	CIFAR-10	ImageNet	CNN builds in
Shift				yes
Scale				no
Small rotation				no
Large rotation				no
Mirror left–right				no
Color change				no

A CNN builds in shift and locality, nothing else. Rotation, scale, mirroring and color still have to come from augmentation, or from networks built for them: group-equivariant CNNs share filters across rotations and mirrors (Cohen & Welling, ICML 2016); scale-equivariant CNNs share them across sizes (Sosnovik et al., ICLR 2020).

QUESTION

Fill in the table. Which transformations leave the label alone in each dataset, and over what range?

A CHECKLIST FOR EACH AUGMENTATION

1. Name the transformation.
2. Check the label survives it **in your data**: a mirrored 3 is not a 3; a mirrored truck is a truck.
3. Augment only over the range your test data really shows.

© 2026 Dr. Jesse Roberts

What does other data ignore?

For each kind of data: which changes keep the label, and which break it?

AUDIO

Speech clip

Which word was said?

keeps: shift in time; a bit louder or faster

breaks: played backwards

built in by: 1-D convolutions over time

3-D STRUCTURE

Molecule

Is it toxic?

keeps: rotate or move it in space; renumber the atoms

breaks: its mirror image (can change biology)

built in by: graph nets that are rotation-equivariant

SETS

Lidar point cloud

Is this a car?

keeps: list the points in any order; turn about the vertical axis

breaks: flip it upside down

built in by: PointNet, Deep Sets (Qi 2017; Zaheer 2017)

GRAPHS

Social network

Is this account a bot?

keeps: relabel the people

breaks: add or cut friendships

built in by: graph neural networks

OVERHEAD IMAGES

Satellite or microscope image

Which crop / which cell type?

keeps: any rotation; mirroring: there is no "up"

breaks: change the scale bar

built in by: group-equivariant CNNs (Cohen & Welling 2016)

TEXT

Review sentence

Positive or negative?

keeps: a synonym swap; padding at the start

breaks: reorder the words: "not bad, good" vs "not good, bad"

built in by: attention ignores order; positions are added back

Every kind of data has its own invariances, and **architectures built to match them**.

© 2026 Dr. Jesse Roberts

What to take away

Steering what a network learns, in three lines

01

Invariances belong to the **task**. Inductive bias belongs to the **learner**. Learning goes best when they match.

02

Augmentation **shows** the invariance through the data. Architecture **assumes** it in the weights.

03

A right bias is **worth data**. A wrong one makes the good models unlikely.

Beyond learning: **choose what the network is likely to learn**.

© 2026 Dr. Jesse Roberts